

BranchCARET Model Checking of Self Modifying Code^{*}

Tayssir Touili and Olzhas Zhangel'dinov

IRIF, CNRS, and University Paris Cité

Abstract. Binary code analysis is an important task, especially for malware detection. Model checking is one of the static analysis techniques that can reason about behaviour of a program. However, when working with binary code, it is crucial to take self-modifying instructions into account. We consider Self-Modifying Pushdown Systems (SM-PDS) as an abstraction for self-modifying binary code. To specify malicious behaviour, LTL, CTL, and CARET have been used previously. However, sometimes, to specify malicious behaviour, one needs both branching features of CTL and call-return relations of CARET. BranchCARET was proposed to combine these features, and several BranchCARET model checking algorithms for standard Pushdown Systems were proposed. In this work, we present an efficient algorithm for BranchCARET model checking of *self-modifying* PDS. This task is not trivial, because BranchCARET allows to reason about past states, while self-modification can change the set of applicable transitions, such that the transitions are not the same in the past and in the present. We tackle this by recording self-modifications on the stack. We have implemented our algorithm in a tool and obtained promising results. In particular, we were able to detect several self-modifying malwares.

1 Introduction

Analysis of binary programs is an important issue when one does not have an access to a program's original source code. This is often the case in malware analysis and analysis of proprietary software. However, binary programs are often obfuscated using self-modification. Therefore, it is important to analyse *binary* programs with *self-modification*.

In this work, we consider verification of self-modifying binary code. First, let us demonstrate an example of self-modifying code in Listing 1.1. This code is supposed to run on an x86 architecture under Windows operating system.

The binary program starts execution at the address `0x14` that contains the bytecode `eb00`. This bytecode corresponds to the assembly instruction `jmp 0x16`, which jumps to the address `0x16`. The address `0x16` contains binary code `c6051500000009`, which corresponds to the assembly instruction `mov [0x15],`

^{*} This work was partially funded by the french ANR grant Defmal “ANR-22-PECY-0007”

#	Address	Bytecode	Assembly
	0x14	eb00	jmp 0x16
	0x16	c6051500000009	mov [0x15], 0x09
	0x1d	ebf5	jmp 0x14
	0x1f	ff1500000000	call [InternetOpenA]

Listing 1.1. Example of self-modifying code.

#	Address	Assembly	
	0x14	jmp 0x16	#-- executing -> jmp 0x1f
			# mov [0x15], 0x09
	0x16	mov [0x15], 0x09	
	0x1d	jmp 0x14	
	0x1f	call [InternetOpenA]	

Listing 1.2. Example from Listing 1.1 after self-modification.

0x09. This instruction replaces the byte stored at address 0x15 with the byte 0x09. Note that the address 0x15 corresponds to the byte 00 inside of the bytecode `eb00` stored at the address 0x14. In other words, the instruction `mov [0x15], 0x09` replaces the bytecode `eb00` with the bytecode `eb09`. The new code `eb09` corresponds to the assembly instruction `jmp 0x1f`, which jumps to the address 0x1f. Thus, the `mov` instruction is a self-modifying instruction that replaces the instruction `jmp 0x16` with the instruction `jmp 0x1f`. After the `mov` instruction, the program executes the address 0x1d, which contains the bytecode `ebf5`, corresponding to the assembly instruction `jmp 0x14`, which jumps to the address 0x14. Now, the address 0x14 contains the freshly modified instruction `jmp 0x1f` instead of the original `jmp 0x16`. The binary code after self-modification is presented in Listing 1.2. The new instruction jumps to the address 0x1f, which contains the bytecode `ff1500000000`. This bytecode corresponds to the instruction `call [InternetOpenA]`, which calls the Windows API function *InternetOpenA*. This function accesses the internet and opens the provided URL. However, if one analyses this binary without considering self-modification, the instruction calling the *InternetOpenA* function would not appear to be reachable. The program would appear to consist of only the infinite loop at addresses 0x14, 0x16, and 0x1d. Therefore, it is important to consider self modification when analysing binary code.

To verify self-modifying binaries, we consider model checking, which allows to reason about behaviour of the program without executing it. Previous studies that consider model checking [17, 23] have successfully proved that binary code can be efficiently model checked for the purpose of malware detection. However, one requires an adequate abstraction for model checking. Pushdown System (PDS) is a natural abstraction for binary programs [23]. To model self-modifying behaviour, we consider Self-Modifying Pushdown Systems (SM-PDS) proposed by Touili and Ye [24], who also proposed reachability analysis, LTL,

and CTL model checking of SM-PDS. However, LTL and CTL is sometimes not enough for malware detection. It is important to reason about calls and their corresponding returns to have understanding of return values and parameters of functions. Temporal Logic for Calls and Returns (CARET) [2] was introduced for this reason. Later, [20] proposed CARET model checking of PDS. However, CARET can reason only about one possible run of the program. And there are malicious behaviours that cannot be described with CARET [21]. Instead, one requires to talk about which branches a program may take. Such expressivity is obtainable only with branching time logics. To solve this, [21] and [16] proposed two branching versions of CARET (BranchCARET) simultaneously. Both of these works propose a new branching time logic that can reason about function calls and returns. However, the authors considered only model checking on standard, non self-modifying Pushdown Systems, which may not be enough in the case of self-modifying code.

We go one step further and present a *direct* and *efficient* algorithm for BranchCARET model checking of SM-PDS, which is a natural abstraction for self-modifying binary code. To perform model checking and compute whether the SM-PDS $\mathcal{P}$ satisfies a BranchCARET formula ψ , we follow the lines of [21] and [16]. We construct a kind of alternating Büchi pushdown system $\mathcal{BP}$, such that there is an infinite run in $\mathcal{BP}$ that visits accepting control locations infinitely many times only if $\mathcal{P}$ satisfies ψ . This is not trivial for two reasons. First, BranchCARET can reason about *abstract* successors, which refer to the next step in the *same function*. I.e., if the current step is a call, then the *abstract* successor ‘steps over’ the inner call and refers directly to the return location. Although this problem was solved for standard non self-modifying PDS, the problem is harder for SM-PDS. Not only one needs to know the return location, but also, if a self-modification happened during the inner call, then the return location must reflect the self-modification. The second reason is the *caller* successor in BranchCARET. This kind of successors refers to the latest call *in the past*. Again, to reason about the past state, one needs to know if a self-modification happened since this past state. We solve these two problems by introducing new types of control locations that are activated when reasoning about abstract and caller successors. Moreover, we store information about states in the past inside of special stack symbols that we call *checkpoints*. If ψ requires to ‘go back in time’, these checkpoints can be read from the stack and the old state can be restored. Then, to find accepting runs in $\mathcal{BP}$, we introduce an efficient algorithm that constructs a finite-state automaton $\mathcal{A}$ that accepts configurations from which an accepting run can be constructed.

We have implemented our algorithm in a tool for model checking binary programs. Then, we expressed several malicious behaviour using BranchCARET formulas, such that if a given binary satisfies the formula, then the binary is marked as malware. We then applied the tool on malwares taken from public databases and the tool was able to detect malicious behaviour in many self-modifying malwares.

Related Works Model checking of binary code is an extensively studied field with many approaches. One of the most popular approaches is symbolic execution [18], which consists in representing the state of the program as a set of logical constraints and simulating the execution of the program on the constraints, instead of the concrete program state. Symbolic execution of binaries has been employed in such platforms as BINSEC [14], S2E [11], and Zorya [15]. However, symbolic execution alone is not enough for malware detection, as one needs a robust way to specify features of malware.

One way of describing malicious behaviour is temporal logic, which was used by numerous studies [4–6, 17, 20, 23] in the context of malware detection. However, these works do not consider self-modification.

Some studies [7, 9, 10, 12] proposed formal models for self-modifying code, but such models do not scale to large real world binaries. Control flow reconstruction of self-modifying binaries was proposed by [3], but this technique ignores execution stack. Other studies [8, 19] provide a technique to disassemble packed binaries. However, these works focus solely on packing behaviour, and not on self-modification in general. Touili and Ye [24] proposed model checking for Self-Modifying Pushdown System (SM-PDS). However, in these works, only reachability, LTL, and CTL model checking is considered. Whereas our approach adopts the SM-PDS model and proposes effective model checking of SM-PDS over BranchCARET.

Outline

2 Preliminaries

2.1 Self-Modifying Pushdown Systems

We define our model for self-modifying binary code:

Definition 1. A Labelled Self-Modifying Pushdown System (SM-PDS) is a tuple $\mathcal{P} = (P, \Gamma, \Delta, \Delta^c)$, such that P is a finite set of control locations, Γ is a stack alphabet, Δ is a finite set of rules of the forms: (1) $p\gamma \xrightarrow{int} p_1\omega_1$, (2) $p\gamma \xrightarrow{call} p_1\gamma_1\gamma_2$, and (3) $p\gamma \xrightarrow{ret} p_1$, where $p, p_1 \in P$, $\gamma, \gamma_1, \gamma_2 \in \Gamma$, $\omega_1 \in \Gamma^*$; Δ^c is a finite set of self-modifying rules of the form $p \xrightarrow{(\rho_1, \rho_2)} p_1$, where $\rho_1, \rho_2 \subseteq \Delta \cup \Delta^c$. A Labelled Pushdown System (PDS) is an SM-PDS, such that $\Delta^c = \emptyset$.

To express the self-modifying behaviour of SM-PDS, we introduce the notion of phase $\theta \subseteq \Delta \cup \Delta^c$, which is the current set of rules that can be applied. Intuitively, a rule of the form $r = p\gamma \xrightarrow{int} p_1\omega_1$ can be applied only if $r \in \theta$. r pops γ from the stack, pushes the word ω_1 onto the stack (ω_1 can be empty), and changes the current control location from p to p_1 . A rule of the form $p\gamma \xrightarrow{call} p_1\gamma_1\gamma_2$ often corresponds to a call statement of the form $\gamma \xrightarrow{call \text{ } proc} \gamma_1$, where *proc* is the name of the called procedure, γ is the location of the call statement, γ_1 is the entry location of *proc*, γ_2 is the return location, and p, p_1 represent global state

of the program. A rule of the form $p\gamma \xrightarrow{ret} p_1$ corresponds to a return statement. A self-modifying rule of the form $p \xrightarrow{(\rho_1, \rho_2)} p_1$ changes the control location from p to p_1 , removes every rule of ρ_1 from the current phase, and adds every rule from ρ_2 to the current phase.

For an SM-PDS $\mathcal{P} = (P, \Gamma, \Delta, \Delta^c)$, a *configuration* $c \in Conf = P \times \Gamma^* \times 2^{\Delta \cup \Delta^c}$ of $\mathcal{P}$ is a tuple of the form $(\langle p, \omega \rangle, \theta)$, where $p \in P$ is the current control location, $\omega \in \Gamma^*$ is the current stack content, and $\theta \subseteq \Delta \cup \Delta^c$ is the current phase. For configurations c, c' , and a label $t \in \{call, int, ret\}$, the successor relation $c \xRightarrow{t} c'$ is defined as follows. For any $p, p' \in P$, $\omega \in \Gamma^*$, and $\theta, \theta', \rho, \rho' \subseteq \Delta \cup \Delta^c$:

1. if there is a rule $r = p\gamma \xrightarrow{int} p'\omega' \in \theta$, then $(\langle p, \gamma\omega \rangle, \theta) \xRightarrow{int} (\langle p', \omega'\omega \rangle, \theta)$,
2. if there is a rule $r = p\gamma \xrightarrow{call} p'\gamma_1\gamma_2 \in \theta$, then $(\langle p, \gamma\omega \rangle, \theta) \xRightarrow{call} (\langle p', \gamma_1\gamma_2\omega \rangle, \theta)$,
3. if there is a rule $r = p\gamma \xrightarrow{ret} p' \in \theta$, then $(\langle p, \gamma\omega \rangle, \theta) \xRightarrow{ret} (\langle p', \omega \rangle, \theta)$,
4. if there is a rule $r = p \xrightarrow{(\rho, \rho')} p' \in \theta$ and $\rho \subseteq \theta$, then $(\langle p, \omega \rangle, \theta) \xRightarrow{int} (\langle p', \omega \rangle, \theta')$, where $\theta' = (\theta \setminus \rho) \cup \rho'$.

A *run* π on $\mathcal{P}$ is an infinite sequence of configurations of the form $\pi = c_0c_1\ldots$, where for every $i \geq 0$, $c_i \in Conf$, and $\exists t_i \in \{call, int, ret\}$, s.t. $c_i \xRightarrow{t_i} c_{i+1}$. We denote by $\pi(i)$ be the i -th configuration c_i of π . Let $Ext(\pi, i)$ be the set of runs on $\mathcal{P}$, s.t. for each $\pi' \in Ext(\pi, i)$, π' is an extension of π from i , i.e. for each $j \leq i$, $\pi'(j) = \pi(j)$.

2.2 BranchCARET formulas

In this section, we define BranchCARET formulas. Given a set of atomic propositions AP , we define a BranchCARET formula ψ over AP (we assume that BranchCARET formulas are given in NNF):

Definition 2. A BranchCARET formula ψ is ($a \in AP$ and $b \in \{g, a, c\}$):

$$\begin{aligned} \psi ::= & a \mid \neg a \mid \psi \vee \psi \mid \psi \wedge \psi \mid EX^b\psi \mid AX^b\psi \\ & \mid E[\psi U^b\psi] \mid A[\psi U^b\psi] \mid E[\psi R^b\psi] \mid A[\psi R^b\psi] \end{aligned}$$

For an SM-PDS $\mathcal{P} = (P, \Gamma, \Delta, \Delta^c)$ and a BranchCARET formula ψ , let λ be a labelling function $\lambda : P \rightarrow 2^{AP}$ that labels each control location with a set of atomic propositions. Let $\pi = c_0c_1\ldots$ be a run on $\mathcal{P}$, such that for $i, i \geq 0$, $\pi(i) = c_i = (\langle p_i, \omega_i \rangle, \theta_i)$, where $p_i \in P$, $\omega_i \in \Gamma^*$, $\theta_i \subseteq \Delta \cup \Delta^c$. For any $i \geq 0$, the configuration c_i at index i on π may have three types of successors:

- the *global successor*, $next_\pi^g(i) = i + 1$;
- the *abstract successor*, $next_\pi^a(i)$, depends on the transition $c_i \xRightarrow{t_i} c_{i+1}$:
 - if $t_i = call$, then $next_\pi^a(i)$ is the index of the matching *ret*,

- if $t_i = \text{int}$, then $\text{next}_\pi^a(i) = i + 1$, and
- if $t_i = \text{ret}$, then $\text{next}_\pi^a(i) = \perp$;
- the *caller successor*, $\text{next}_\pi^c(i)$, is the index of the nearest unmatched *call* ($\text{next}_\pi^c(i) < i$), otherwise $\text{next}_\pi^c(i) = \perp$.

Note that sometimes $\pi(i)$ might not have an abstract (resp. caller) successor. In these cases, we say that $\text{next}_\pi^a(i)$ (resp. $\text{next}_\pi^c(i)$) is undefined, or $\text{next}_\pi^a(i) = \perp$ (resp. $\text{next}_\pi^c(i) = \perp$).

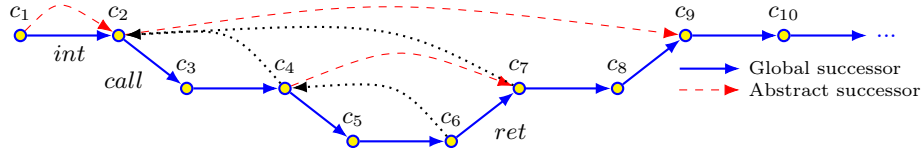


Fig. 1. Types of successors in a run π' on an SM-PDS.

For example, a run $\pi' = c_1 c_2 c_3 \dots$ is illustrated in Figure 1. Three different types of successors are signified by different arrows. Solid arrows show *global successors* and dashed arrows show *abstract successors*. For any $i \geq 1$, a configuration c_i may have three types of successors. The global successor is always c_{i+1} . The abstract and the global successor of c_1 is c_2 : $\text{next}_{\pi'}^g(1) = \text{next}_{\pi'}^a(1) = 2$. Configurations c_8 and c_6 do not have abstract successors: $\text{next}_{\pi'}^a(6) = \text{next}_{\pi'}^a(8) = \perp$. The abstract successor of c_2 is c_9 , which is the corresponding return of c_2 : $\text{next}_{\pi'}^a(2) = 9$.

We write $(\pi, i) \models \psi$ if π satisfies ψ at the i -th step. The satisfaction is defined as follows:

1. $(\pi, i) \models e$, where $e \in AP$, if $e \in \lambda(p_i)$;
2. $(\pi, i) \models \neg e$, where $e \in AP$, if $e \notin \lambda(p_i)$;
3. $(\pi, i) \models \psi_1 \vee \psi_2$ if $(\pi, i) \models \psi_1$ or $(\pi, i) \models \psi_2$;
4. $(\pi, i) \models \psi_1 \wedge \psi_2$ if $(\pi, i) \models \psi_1$ and $(\pi, i) \models \psi_2$;
5. $(\pi, i) \models EX^b \psi$, where $b \in \{g, a, c\}$, if there exists $\pi' \in \text{Ext}(\pi, i)$, s.t. $\text{next}_{\pi'}^b(i) \neq \perp$ and $(\pi', \text{next}_{\pi'}^b(i)) \models \psi$;
6. $(\pi, i) \models AX^b \psi$, where $b \in \{g, a, c\}$, if for all $\pi' \in \text{Ext}(\pi, i)$, $\text{next}_{\pi'}^b(i) \neq \perp$ implies $(\pi', \text{next}_{\pi'}^b(i)) \models \psi$;
7. $(\pi, i) \models E[\psi_1 U^b \psi_2]$, where $b \in \{g, a, c\}$, if there exists $\pi' \in \text{Ext}(\pi, i)$, s.t. there is a sequence $s = s_0 s_1 \dots s_k$, where $s_0 = i$, for j , $0 \leq j < k$, $s_{j+1} = \text{next}_{\pi'}^b(s_j)$, $(\pi', s_j) \models \psi_1$, and $(\pi', s_k) \models \psi_2$;
8. $(\pi, i) \models A[\psi_1 U^b \psi_2]$, where $b \in \{g, a, c\}$, if for all $\pi' \in \text{Ext}(\pi, i)$, there is a sequence $s = s_0 s_1 \dots s_k$, where $s_0 = i$, for j , $0 \leq j < k$, $s_{j+1} = \text{next}_{\pi'}^b(s_j)$, $(\pi', s_j) \models \psi_1$, and $(\pi', s_k) \models \psi_2$;
9. $(\pi, i) \models E[\psi_1 R^b \psi_2]$, where $b \in \{g, a, c\}$, if there exists $\pi' \in \text{Ext}(\pi, i)$, s.t. for a sequence $s = s_0 s_1 \dots$, where $s_0 = i$ and for j , $j \geq 0$, $s_{j+1} = \text{next}_{\pi'}^b(s_j)$, we have that $(\pi', s_j) \not\models \psi_2$ implies that there is $k < j$, s.t. $(\pi', s_k) \models \psi_1$;

10. $(\pi, i) \models A[\psi_1 R^b \psi_2]$, where $b \in \{g, a, c\}$, if for all $\pi' \in Ext(\pi, i)$ and a sequence $s = s_0 s_1 \dots$, s.t. $s_0 = i$ and for $j, j \geq 0$, $s_{j+1} = next_{\pi'}^b(s_j)$, we have that $(\pi', s_j) \not\models \psi_2$ implies that there is $k < j$, s.t. $(\pi', s_k) \models \psi_1$.

We say that $\pi \models \psi$ if $(\pi, 0) \models \psi$. For a configuration c of $\mathcal{P}$, $c \models \psi$ iff there exists a run π on $\mathcal{P}$ starting from c , s.t. $\pi \models \psi$. Let $Cl(\psi)$ be the closure of ψ , i.e. the set of all subformulas of ψ .

3 BranchCARET Model Checking of SM-PDSs

This section presents our contribution. We propose a direct algorithm for model checking SM-PDSs against BranchCARET formulas. We fix an SM-PDS $\mathcal{P} = (P, \Gamma, \Delta, \Delta^c)$, a BranchCARET formula ψ defined over a set of atomic propositions AP with a labelling function λ .

Before explaining the model checking procedure, let us normalize the representation of SM-PDS rules in Δ and Δ^c by introducing the set τ , defined as:

$$\begin{aligned} \tau = & \{ p\gamma \xrightarrow[\rightarrow_t]{(\{r\}, \{r\})} p'w \mid r = p\gamma \xrightarrow{t} p'w \in \Delta \text{ for } t \in \{int, call, ret\} \} \cup \\ & \{ p\gamma \xrightarrow[\rightarrow_{int}]{(\rho \cup \{r\}, \rho' \cup (\{r\} \setminus \rho))} p'\gamma \mid r = p \xrightarrow{(\rho, \rho')} p' \in \Delta^c, \gamma \in \Gamma \} \end{aligned}$$

Intuitively, a rule of the form $r = p\gamma \xrightarrow{int} p'w \in \Delta$ can be seen as a self-modifying rule that removes $\{r\}$ from the current phase and adds $\{r\}$ to the current phase. This rule can be applied only if $\{r\}$ is in the current phase. Moreover, a self-modifying rule of the form $r = p \xrightarrow{(\rho, \rho')} p' \in \Delta^c$ can be seen as a standard rule that pops γ from the stack and pushes γ back. Moreover, r can be applied only if both $\{r\}$ and ρ are in the current phase. Thus, the self-modification can be seen as removing $\{r\} \cup \rho$ from the current phase and adding $(\{r\} \setminus \rho) \cup \rho'$ to the current phase (note that if $r \in \rho$, then r should not be added, so we say $\{r\} \setminus \rho$ is added). Thus, we represent rules in the form $p\gamma \xrightarrow[\rightarrow_t]{(\rho, \rho')} pw \in \tau$, and such rules can be applied only if ρ is in the current phase. Note that we drop the condition that the rule itself is in the phase and instead encode it directly into ρ . This improves readability of the next sections. Now, let us express the global successor relation $\Longrightarrow$ of $\mathcal{P}$ in terms of τ instead of Δ and Δ^c :

- For a configuration $(\langle p, \gamma\omega \rangle, \theta)$, if $\exists p\gamma \xrightarrow[\rightarrow_t]{(\rho, \rho')} p'w \in \tau$, s.t. $\rho \subseteq \theta$, then $(\langle p, \gamma\omega \rangle, \theta) \xrightarrow{t} (\langle p', w\omega \rangle, (\theta \setminus \rho) \cup \rho')$.

Intuitively, $p\gamma \xrightarrow[\rightarrow_t]{(\rho, \rho')} p'w$ means that $\mathcal{P}$ can change the control location from p to p' , pop γ from the stack, remove rules in ρ from the current phase, add rules from ρ' to the current phase, and push w to the stack.

Without loss of generality, we assume that Γ contains a symbol $\#$ that is always located at the bottom of the stack and can never be popped.

3.1 Self-Modifying Alternating Büchi Pushdown System with Checkpoints

Model checking of SM-PDS against BranchCARET formulas is not trivial and more complex than BranchCARET model checking of standard PDS. Before introducing the complete algorithm, let us first present the challenges one faces during model checking.

The common approach to model checking SM-PDS over branching time logic formulas is to construct an SM-ABPDS:

Definition 3. [25] *A Self-Modifying Alternating Büchi Pushdown System (SM-ABPDS) is a tuple $\mathcal{BP} = (P', \Gamma', \Delta', F, \delta)$, where P' is a set of control locations, Γ' is the stack alphabet, F is the set of accepting control locations, and $\delta = \Delta \cup \Delta^c$ is the phase alphabet, Δ' is a set of alternating rules of the form $p\gamma \xrightarrow{[\sigma_1, \dots, \sigma_h]} \{p_1w_1, \dots, p_hw_h\}$, where $\sigma_i = (\rho_i, \rho'_i) \in 2^\delta \times 2^\delta$.*

An Alternating Büchi Pushdown System (ABPDS) [23] is an SM-ABPDS $\mathcal{BP}' = (P', \Gamma', \Delta', F, \delta)$, such that $\delta = \emptyset$.

SM-ABPDS is a kind of extension of standard ABPDS that can also express self-modifying behaviour. SM-ABPDS needs to keep track of the phase in the original SM-PDS, therefore we define the elements of phases in SM-ABPDS (phase alphabet) to be $\delta = \Delta \cup \Delta^c$ - the original rules in $\mathcal{P}$. Moreover, application of rules in SM-ABPDS depends on what symbols from δ are present in the current phase. A configuration in an SM-ABPDS $\mathcal{BP} = (P', \Gamma', \Delta', F, \delta)$ is a tuple of the form $(\langle p, w \rangle, \theta)$, where $p \in P'$, $w \in \Gamma'^*$, and $\theta \subseteq \delta$.

The successor relation $c \Longrightarrow S$ of a configuration c and a set of configurations S of an SM-ABPDS $\mathcal{BP}$ is defined as follows:

- if $\exists p\gamma \xrightarrow{[\sigma_1, \dots, \sigma_h]} \{p_1w_1, \dots, p_hw_h\}$, then for any $\omega \in \Gamma'^*$, $\theta \subseteq \delta$, $(\langle p, \gamma\omega \rangle, \theta) \Longrightarrow \{(\langle p_i, w_i\omega \rangle, (\theta \setminus \rho_i) \cup \rho'_i) \mid \sigma_i = (\rho_i, \rho'_i), \rho_i \subseteq \theta\}$.

Intuitively, the rules of SM-ABPDS correspond to application of multiple rules in the original SM-PDS. σ -labels represent two things - which rules must be present in the current phase for the rule to take place, and the self-modifying behaviour (in the same fashion as in τ -rules). A run η of an SM-ABPDS $\mathcal{BP}$ is a tree, such that each node is labelled with a configuration c of $\mathcal{BP}$ and if it has the set of children S , then $c \Longrightarrow S$. The run η is accepting iff each branch visits accepting control locations from F infinitely many times. For some rule $p\gamma \xrightarrow{[\sigma_1, \dots, \sigma_h]} \{p_1w_1, \dots, p_hw_h\}$ in SM-ABPDS $\mathcal{BP}$, if for $1 \leq i \leq h$, $\sigma_i = (\emptyset, \emptyset)$, for succinctness, we say $\sigma_i = \top$, i.e. that the i -th branch does not require any rule to be in the current phase, and the phase is not changed after the transition.

A naive approach to BranchCARET model checking of SM-PDS is to construct an SM-ABPDS $\mathcal{BP}$, such that if some configuration $(\langle p, \gamma\omega \rangle, \theta)$ satisfies ψ in $\mathcal{P}$, then there is an accepting run in $\mathcal{BP}$ starting from $(\langle [p, \psi], \gamma\omega \rangle, \theta)$ (the subformulas of ψ are encoded into the control locations of $\mathcal{BP}$). As we will show

in this section, this process gets complicated because besides standard CTL operators, BranchCARET also reasons about abstract and caller successors. The caller successors are hard to deal with because they are located in the relative past of the configuration that, e.g., needs to be checked against $EX^c\phi$. The abstract successors are not straightforward neither because abstract successors of calls are not located right after the call, but instead at some state of the corresponding return, which is unknown beforehand. To tackle the caller successors, we introduce a special type of control locations and stack symbols that allow the SM-ABPDS to ‘roll back’ to the past and obtain the caller successors. To tackle abstract successors, we introduce another type of control locations and stack symbols that instruct the SM-ABPDS to ‘roll forward’ to return locations to find abstract successors of calls.

First, consider we are given a configuration $(\langle p, \gamma\omega \rangle, \theta)$ of an SM-PDS $\mathcal{P}$ and we would like to check if it satisfies a standard CTL formula $\psi = AX^g\phi$.

Suppose $\{p\gamma \xrightarrow{(\rho_1, \rho'_1)}_{int} p_1w_1, \dots, p\gamma \xrightarrow{(\rho_h, \rho'_h)}_{int} p_hw_h\}$ are all rules in τ with $p\gamma$ as the left-hand side. Therefore, for $1 \leq i \leq h$, the i -th successor of $(\langle p, \gamma\omega \rangle, \theta)$ in $\mathcal{P}$ has the form $(\langle p_i, w_i\omega \rangle, \theta_i)$, where $\theta_i = (\theta \setminus \rho_i) \cup \rho'_i$. Additionally, the i -th successor is valid only if $\rho_i \subseteq \theta$. In $\mathcal{BP}$, these rules will correspond to a rule of the form $[p, AX^g\phi]\gamma \xrightarrow{[\sigma_1, \dots, \sigma_h]} \{[p_1, \phi]w_1, \dots, [p_h, \phi]w_h\}$, where $\sigma_i = (\rho_i, \rho'_i)$ for $1 \leq i \leq h$. Therefore, in $\mathcal{BP}$, there will be a transition $(\langle [p, AX^g\phi], \gamma\omega \rangle, \theta) \Longrightarrow \{(\langle [p_i, \phi], w_i\omega \rangle, \theta_i) \mid \rho_i \subseteq \theta\}$, and thus, there will be an accepting run in $\mathcal{BP}$ if there are accepting runs from each child $(\langle [p_i, \phi], w_i\omega \rangle, \theta_i)$. By the nature of the construction of $\mathcal{BP}$, if there is an accepting run from $(\langle [p_i, \phi], w_i\omega \rangle, \theta_i)$ in $\mathcal{BP}$, then $(\langle p_i, w_i\omega \rangle, \theta_i) \models \phi$. And since $(\langle p_i, w_i\omega \rangle, \theta_i)$ is the i -th successor of $(\langle p, \gamma\omega \rangle, \theta)$ in $\mathcal{P}$, then there is an accepting run from $(\langle [p, AX^g\phi], \gamma\omega \rangle, \theta)$ in $\mathcal{BP}$ only if each successor in $\mathcal{P}$ satisfies ϕ .

However, such intuition is not enough for BranchCARET model checking because of *caller* modalities, which look into the *past* configurations. Suppose $\psi = AX^gEX^c\phi$, and there is a single rule $r = p\gamma \xrightarrow{call} p_1\gamma_1\gamma'_1$ with $p\gamma$ on the left-hand side in Δ (i.e. $p\gamma \xrightarrow{(\{r\}, \{r\})}_{call} p_1\gamma_1\gamma'_1 \in \tau$). If we apply the same intuition here, we will obtain a rule $r' = [p, AX^gEX^c\phi]\gamma \xrightarrow{[(\{r\}, \{r\})]} \{[p_1, EX^c\phi]\gamma_1\gamma'_1\}$ in $\mathcal{BP}$, and thus, we will have that there is an accepting run from $(\langle [p, AX^gEX^c\phi], \gamma\omega \rangle, \theta)$ only if there is an accepting run from $(\langle [p_1, EX^c\phi], \gamma_1\gamma'_1\omega \rangle, \theta)$. Now we need to determine if there is an accepting run, let us call it η , starting from $(\langle [p_1, EX^c\phi], \gamma_1\gamma'_1\omega \rangle, \theta)$. And since the caller successor of $(\langle p_1, \gamma_1\gamma'_1\omega \rangle, \theta)$ in $\mathcal{P}$ is $(\langle p, \gamma\omega \rangle, \theta)$, finding η should be equivalent to finding if $(\langle p, \gamma\omega \rangle, \theta) \models \phi$, and consequently, to finding if there is an accepting run η' starting from $(\langle [p, \phi], \gamma\omega \rangle, \theta)$. In other words, our $\mathcal{BP}$ must be such that η exists only if η' exists. We follow the approach of [16] and encode *call histories* into the stack. Instead of rule r' in $\mathcal{BP}$, we construct rules of the following forms:

1. $[p, AX^gEX^c\phi]\gamma \hookrightarrow \{[p_1, EX^c\phi]\gamma_1[p, \gamma]\}$,
2. $[p_1, EX^c\phi]\gamma_1 \hookrightarrow \{[p^c, EX^c\phi]\varepsilon\}$, where p^c is a new control location, and
3. $[p^c, EX^c\phi]\varepsilon \hookrightarrow \{[p, \phi]\gamma\}$,

Where $[\gamma'_1, p, \gamma]$ is the stack symbol γ'_1 encoded with the *call history*, i.e. the control location p and γ because the configuration in $\mathcal{P}$ before the call was $(\langle p, \gamma \omega \rangle, \theta)$. Hence, these rules create the following path: $(\langle p, AX^g EX^c \phi \rangle, \gamma \omega, \theta) \Rightarrow^1 (\langle p_1, EX^c \phi \rangle, \gamma_1 [\gamma'_1, p, \gamma] \omega, \theta) \Rightarrow^2 (\langle p^c, EX^c \phi \rangle, [\gamma'_1, p, \gamma] \omega, \theta) \Rightarrow^3 (\langle p, \phi \rangle, \gamma \omega, \theta)$, where $\Rightarrow^i$ means that the i -th rule was applied to get the transition. Thus, in such $\mathcal{BP}$, η exists if η' exists.

However, this is still not enough when we consider self-modification. Suppose we have an SM-PDS $\mathcal{P}$ as in Figure 2, and we want to determine if $(\langle p_3, \gamma_2 \gamma_3 \rangle, \theta_2)$ satisfies formula $EX^c \phi$, given we start from $(\langle p_1, \gamma_1 \rangle, \theta_1)$. Figure 2 demonstrates what successor transitions $\mathcal{P}$ can have. More specifically, it shows that because r_2 changes the current phase from θ_1 to θ_2 , then the caller successor of $(\langle p_3, \gamma_2 \gamma_3 \rangle, \theta_2)$ is $(\langle p_1, \gamma_1 \rangle, \theta_1)$, and not $(\langle p_1, \gamma_1 \rangle, \theta_2)$ (despite having the same phase θ_2).

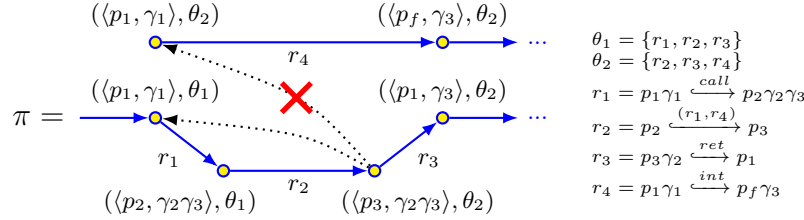


Fig. 2. Types of successors in a run π' on an SM-PDS.

With the naive approach, one could encode $p_1 \gamma_1$ into the return location γ_3 , such that the corresponding configuration in the resulting SM-ABPDS $\mathcal{BP}$ is $(\langle p_3, EX^c \phi \rangle, \gamma_2 [\gamma_3, p_1, \gamma_1], \theta_2)$. If we apply the same logic as for standard PDS, then $\mathcal{BP}$ will transition into $(\langle p_1, \phi \rangle, \gamma_1, \theta_2)$, which is wrong, because the correct caller successor contains phase θ_1 and not θ_2 . Figure 2 demonstrates why we cannot assume that the caller successor has the same phase. If we did, then π would satisfy completely different formulas.

To tackle this, we extend the semantics of *call histories* and define a new type of stack symbols, called *checkpoints*, which are special symbols that are embedded with the current phase when pushed onto the stack. We denote the alphabet of checkpoints as X , and its items as χ . Now, since $\mathcal{P}$ in Figure 2 contains the rule $p_1 \gamma_1 \xrightarrow{\text{call}} p_2 \gamma_2 \gamma_3$, it will correspond to a rule of the form $r' = [p_1, AX^g \phi'] \gamma_1 \hookrightarrow [p_2, \phi'] \gamma_2 \chi$ in $\mathcal{BP}$ for some $AX^g \phi'$, where $\chi = [\gamma_3, p_1, \gamma_1]$ is embedded with the call history. Moreover, when χ is pushed to the stack, $\mathcal{BP}$ automatically embeds χ with the current phase. For example, rule r' will create the transition $(\langle p_1, AX^g \phi' \rangle, \gamma_1, \theta_1) \Rightarrow (\langle p_2, \phi' \rangle, \gamma_2 [\gamma_3, p_1, \gamma_1]^{\theta_1}, \theta_1)$, where the checkpoint $[\gamma_3, p_1, \gamma_1]^{\theta_1}$ is embedded with the current phase θ_1 at the call location. Thus, when $\mathcal{BP}$ reaches $(\langle p_3, EX^c \phi \rangle, \gamma_2 [\gamma_3, p_1, \gamma_1]^{\theta_1}, \theta_2)$, applies a rule of the form $[p_3, EX^c \phi] \gamma_2 \hookrightarrow \{[p^c, \phi]\}$, and pops γ_2 from the stack,

it will be at configuration $(\langle [p^c, \phi], [\gamma_3, p_1, \gamma_1]^{\theta_1} \rangle, \theta_2)$. Then, we define special *restore* rule of the form $[p^c, \phi][\gamma_3, p_1, \gamma_1] \xrightarrow{\text{restore}} [p_1, \phi]\gamma_1$, which decodes χ and changes the current phase to the phase χ was encoded with. I.e. it produces transition $(\langle [p^c, \phi], [\gamma_3, p_1, \gamma_1]^{\theta_1} \rangle, \theta_2) \Rightarrow (\langle [p_1, \phi], \gamma_1 \rangle, \theta_1)$, where $(\langle [p_1, \phi], \gamma_1 \rangle, \theta_1)$ is the correct caller successor. Note that if $\mathcal{P}$ executes a return rule r_3 in Figure 2, then in the corresponding SM-ABPDS $\mathcal{BP}$, the checkpoint χ will be on top of the stack in the configuration of the form $(\langle [p_1, \phi][\gamma_3, p_1, \gamma_1]^{\theta_1} \rangle, \theta_2)$. In this case, $\mathcal{BP}$ does not need to restore the call history. Instead, it needs to just discard the history and continue the execution corresponding to the execution of $\mathcal{P}$. Thus, we define a *discard* rule of the form $[p_1, \phi][\gamma_3, p_1, \gamma_1] \xrightarrow{\text{discard}} [p_1, \phi]\gamma_3$, which discards the call history and the encoded phase and creates a transition of the form $(\langle [p_1, \phi], [\gamma_3, p_1, \gamma_1]^{\theta_1} \rangle, \theta_2) \Rightarrow (\langle [p_1, \phi], \gamma_3 \rangle, \theta)$. Such rule should be applied when $\mathcal{BP}$ does not reason about the caller successor, i.e. at control locations of the form $[p, \phi]$, where $p \neq p^c$.

On the other hand, consider a formula $\psi = EX^a\phi$ and the SM-PDS illustrated on Figure 2. Suppose we want to check $(\langle p_1, \gamma_1 \rangle, \theta_1) \models \psi$. The abstract successor of $(\langle p_1, \gamma_1 \rangle, \theta_1)$ is $(\langle p_1, \gamma_3 \rangle, \theta_2)$, so ψ needs to be carried over to $(\langle p_1, \gamma_3 \rangle, \theta_2)$. To achieve this, we encode ψ into $(\langle [p_1, \psi], \gamma_1 \rangle, \theta_1)$ and from rule r_1 in $\mathcal{P}$, we construct a rule of the form $[p_1, EX^a\phi]\gamma_1 \xrightarrow{(\{r_1\}, \{r_1\})} [p_2, EX^a\phi]^r\gamma_2\gamma_3^r$. We use the control location $[p_2, EX^a\phi]^r$ to express that this control location does not actually check $EX^a\phi$, but instead looks for a corresponding return to check ϕ at the return state (at the abstract successor). We mark γ_3^r by r meaning that when this symbol is on top of the stack, the corresponding return has happened. Then, we construct rules that corresponds to r_2 and r_3 : $[p_2, \psi]^r\gamma_2 \xrightarrow{[(\{r_2, r_1\}, \{r_2, r_4\})]} \{[p_3, \psi]^r\gamma_2\}$ and $[p_3, \psi]^r\gamma_2 \xrightarrow{[(\{r_3\}, \{r_3\})]} \{[p_1, \psi]^r\epsilon\}$. Thus, applying these rules will give us the path of the form $(\langle [p_1, EX^a\phi], \gamma_1 \rangle, \theta_1) \Rightarrow (\langle [p_2, EX^a\phi]^r, \gamma_2\gamma_3^r \rangle, \theta_1) \Rightarrow (\langle [p_3, EX^a\phi]^r, \gamma_2\gamma_3^r \rangle, \theta_2) \Rightarrow (\langle [p_1, EX^a\phi]^r, \gamma_3^r \rangle, \theta_2)$. Now, it is only necessary to decode the pair $[p_1, EX^a\phi]^r\gamma_3^r$ to obtain the abstract successor. We achieve this using a rule of the form $[p_1, EX^a\phi]^r\gamma_3^r \hookrightarrow [p_1, \phi]\gamma_3$. Therefore $\mathcal{BP}$ will transition into $(\langle [p_1, \phi], \gamma_3 \rangle, \theta_2)$, which corresponds to checking if the abstract successor $(\langle p_1, \gamma_3 \rangle, \theta_2)$ satisfies ϕ .

Note that in case of $\phi = AX^a\phi'$, $\mathcal{BP}$ needs to find all possible return locations. Therefore, it must take every possible path at the same time. For example, if for a control location p and top of the stack γ in $\mathcal{P}$, $\{p\gamma \xrightarrow{\sigma_1}_{t_1} p_1w_1, \dots, p\gamma \xrightarrow{\sigma_h}_{t_h} p_hw_h\}$ are all rules in τ with $p\gamma$ on the left-hand side, these rules will correspond to a single rule of the form $[p, AX^a\phi']^r\gamma \xrightarrow{[\sigma_1, \dots, \sigma_h]} \{[p_1, AX^a\phi']^rw_1, \dots, [p_h, AX^a\phi']^rw_h\}$ in $\mathcal{BP}$.

To sum up, we introduce two different types of control locations and two types of stack symbols. Intuitively, control locations of the form $[p^c, \phi]$ mean that currently, $\mathcal{BP}$ attempts to rollback to the caller successor and then will check if the caller successor satisfies ϕ . To find the caller successor, it pops symbols from the stack until it reaches a *checkpoint* of the form $[\gamma, p_0, \gamma_0]^{\theta_0}$, where p_0 is the

control location at the call site, γ_0 is the topmost symbol on the stack at the call site, and θ_0 is the phase at the call site.

On the other hand, control locations of the form $[p, \phi]^r$ mean that currently, $\mathcal{BP}$ attempts to find the corresponding return location and then will check if the state of $\mathcal{P}$ at the return location satisfies ϕ . The return locations in $\mathcal{BP}$ are of the form γ^r , where γ is the actual return location in $\mathcal{BP}$ and r is a special mark.

Thus, considering the complications described above, we define Self-Modifying Alternating Büchi Pushdown Systems with Checkpoints:

Definition 4. A Self-Modifying Alternating Büchi Pushdown System with Checkpoints is a tuple $\mathcal{BP} = (P', \Gamma', X, \Delta', \text{restore}, \text{discard}, \delta, F)$, where P' is a finite set of control locations, Γ' is a stack alphabet, X is a checkpoint alphabet, Δ' is a set of alternating rules, restore is a set of rules that restore phase from a checkpoint, discard is a set of rules that discard phases from checkpoints, δ is a phase alphabet, and F is an accepting set of control locations.

Rules in Δ' have the following form: $p\gamma \xrightarrow{[\sigma_1, \dots, \sigma_h]} \{p_1w_1, \dots, p_hw_h\}$, where $p \in P'$, $\gamma \in \Gamma$, for $1 \leq i \leq h$, σ_i has the form $(\rho_i, \rho'_i) \in 2^\delta \times 2^\delta$, $p_i \in P'$, and either $w_i \in \Gamma'^*$ or $w_i \in \Gamma' \times X$.

Rules in restore and discard have the form $p\chi \hookrightarrow p'\gamma$, where $p, p' \in P'$, $\chi \in X$, and $\gamma \in \Gamma'$.

Definition 5. A Self-Modifying Alternating Büchi Pushdown System is a Self-Modifying Alternating Büchi Pushdown System with Checkpoints of the form $\mathcal{BP} = (P', \Gamma', X, \Delta', \text{restore}, \text{discard}, \delta, F)$, where $X = \emptyset$, $\text{restore} = \emptyset$, and $\text{discard} = \emptyset$.

From now on, the SM-ABPDS abbreviation will refer to Self-Modifying Alternating Büchi Pushdown Systems with Checkpoints. Consider an SM-ABPDS $\mathcal{BP} = (P', \Gamma', X, \Delta', \text{restore}, \text{discard}, \delta, F)$. When a checkpoint $\chi \in X$ is pushed to the stack, it is embedded with the current phase θ , so that it can be either restored or discarded later. For example, consider a rule $p\gamma \xrightarrow{[\sigma_1, \dots, \sigma_h]} \{p_1w_1, \dots, p_hw_h\}$. This rule is applied when the current control location is p and the topmost symbol on the stack is $\gamma \in \Gamma'$. For $1 \leq i \leq h$, let $\sigma_i = (\rho_i, \rho'_i)$. The i -th branch is taken only if $\rho_i \subseteq \theta$, and when the rule takes place, this branch removes rules in ρ_i from θ and adds rules from ρ'_i to θ . Then, it moves to the control location p_i , and replaces γ on top of the stack with w_i . Moreover, if w_i has the form $\gamma_i\chi \in \Gamma' \times X$, then χ is embedded with θ , i.e. the rule will push $\gamma_i\chi^\theta$ onto the stack. Then, χ^θ can only be popped from the stack using either restore or discard rules. Consider rule $p\chi \hookrightarrow p'\gamma \in \text{restore}$. This rule changes the control location from p to p' , pops χ^θ from the stack, changes the current phase to θ , and pushes γ onto stack. In other words, it restores the phase that was saved when χ was pushed onto stack. On the other hand, a rule of the form $p\chi \hookrightarrow p'\gamma \in \text{discard}$ does the opposite. It changes the control location from p to p' , pops χ^θ from the stack, pushes γ onto stack, and keeps the current phase. In other words, it discards the phase saved when χ was pushed onto the stack.

Let us denote the set of possible symbols on the stack by $G = \Gamma' \cup (X \times 2^\delta)$. Then, for clearer syntax, let us define a function $em : 2^\delta \times (\Gamma'^* \cup (\Gamma' \times X)) \rightarrow G^*$, defined as $em(\theta, w) = w$ for $w \in \Gamma'^*$, and $em(\theta, \gamma\chi) = \gamma\chi^\theta$ for $\gamma\chi \in \Gamma' \times X$. The em function embeds the phase θ into the checkpoints of the word w .

A configuration of $\mathcal{BP}$ is a tuple $(\langle p, w \rangle, \theta) \in Conf_{\mathcal{BP}} = P \times G^* \times 2^\delta$. For configuration c and set of configurations $S \subseteq Conf_{\mathcal{BP}}$, we define the successor relation $c \Longrightarrow S$:

1. if $p\gamma \xrightarrow{[\sigma_1, \dots, \sigma_h]} \{p_1w_1, \dots, p_hw_h\} \in \Delta'$, where for $1 \leq i \leq h$, $\sigma_i = (\rho_i, \rho'_i)$, then $(\langle p, \gamma\omega \rangle, \theta) \Longrightarrow \{(\langle p_i, em(\theta, w_i)\omega \rangle, (\theta \setminus \rho_i) \cup \rho'_i) \mid \text{if } \rho_i \subseteq \theta\}$;
2. if $p\chi \hookrightarrow p'\gamma \in \text{restore}$, then $(\langle p, \chi^\theta\omega \rangle, \theta') \Longrightarrow \{(\langle p', \gamma\omega \rangle, \theta)\}$ for any $\theta' \subseteq \delta$;
3. if $p\chi \hookrightarrow p'\gamma \in \text{discard}$, then $(\langle p, \chi^\theta\omega \rangle, \theta') \Longrightarrow \{(\langle p', \gamma\omega \rangle, \theta')\}$ for any $\theta' \subseteq \delta$;

For a rule $p\gamma \xrightarrow{[\sigma_1, \dots, \sigma_h]} \{p_1w_1, \dots, p_hw_h\}$, if for some $1 \leq i \leq h$, $\sigma_i = (\emptyset, \emptyset)$, then we say $\sigma_i = \top$.

3.2 From BranchCARET Model Checking Problem to the Emptiness Problem of SM-ABPDS

Let $Cl^a(\psi) = \{QX^a\phi_1, Q[\phi_1U^a\phi_2], Q[\phi_1R^a\phi_2] \in Cl(\psi) \mid Q \in \{E, A\}\}$ and $Cl^c(\psi) = \{QX^c\phi_1, Q[\phi_1U^c\phi_2], Q[\phi_1R^c\phi_2] \in Cl(\psi) \mid Q \in \{E, A\}\}$. We can construct SM-ABPDS $\mathcal{BP} = (P', \Gamma', X, \Delta', \text{restore}, \text{discard}, \Delta \cup \Delta^c, F)$, where:

- $P' = P'_g \cup P'_a \cup P'_c$, where:
 - $P'_g = P \times Cl(\psi)$ is a set of control locations of the form $[p, \phi]$,
 - $P'_a = P \times Cl^a(\psi) \times \{r\}$ is a set of control locations of the form $[p, \phi]^r$,
 - $P'_c = \{p^c\} \times Cl^c(\psi)$ is a set of control locations of the form $[p^c, \phi]$, s.t. p^c is a new state;
- $\Gamma' = \Gamma \cup (\Gamma \times \{r\})$, s.t. $\Gamma \times \{r\}$ is the set of stack symbols of the form γ^r ;
- $X = \Gamma \times P \times \Gamma$;
- $F = F^g \cup F^a \cup F^c$, where:
 - $F^g = \{[p, \phi] \in P'_g \mid \phi = Q[\phi_1R^g\phi_2] \text{ for } Q \in \{E, A\}\}$,
 - $F^a = \{[p, \phi]^r \in P'_a \mid \phi = Q[\phi_1R^a\phi_2] \text{ or } \phi = AX^a\phi_1 \text{ for } Q \in \{E, A\}\}$,
 - $F^c = \{[p^c, \phi] \in P'_c \mid \phi = Q[\phi_1R^c\phi_2] \text{ or } \phi = AX^c\phi_1 \text{ for } Q \in \{E, A\}\}$.

To improve clarity, we denote by *True* the control location from which there always exists an accepting run, and we denote by *False* the control location from which there never exists an accepting run. The SM-ABPDS rules Δ' , *restore*, and *discard* are defined as follows. For $p \in P$, $\gamma \in \Gamma$, $\phi \in Cl(\psi)$:

1. if $\phi = a \in AP$ and $a \in \lambda(p)$, then $[p, a]\gamma \xrightarrow{[\top]} \{[p, a]\gamma\}$;
2. if $\phi = \neg a \in AP$ and $a \notin \lambda(p)$, then $[p, \neg a]\gamma \xrightarrow{[\top]} \{[p, \neg a]\gamma\}$;
3. if $\phi = \phi_1 \vee \phi_2$, then $[p, \phi]\gamma \xrightarrow{[\top]} \{[p, \phi_1]\gamma\}$ and $[p, \phi]\gamma \xrightarrow{[\top]} \{[p, \phi_2]\gamma\} \in \Delta'$;
4. if $\phi = \phi_1 \wedge \phi_2$, then $[p, \phi_1 \wedge \phi_2]\gamma \xrightarrow{[\top, \top]} \{[p, \phi_1]\gamma, [p, \phi_2]\gamma\}$;

5. if $\phi = EX^g\phi'$, then $[p, EX^g\phi']\gamma \xrightarrow{[(\rho, \rho')]} \{[p', \phi']w\}$, s.t. either $p\gamma \xrightarrow{(\rho, \rho')}_{int} p'w \in \tau$, $p\gamma \xrightarrow{(\rho, \rho')}_{ret} p'w \in \tau$, or $p\gamma \xrightarrow{(\rho, \rho')}_{call} p'\gamma_1\gamma_2$ and $w = \gamma_1[\gamma_2, p, \gamma]$;
6. if $\phi = AX^g\phi'$, then $[p, AX^g\phi']\gamma \xrightarrow{[\sigma_1, \dots, \sigma_h]} \{[p_1, \phi']w_1, \dots, [p_h, \phi']w_h\}$, s.t. $\{p\gamma \xrightarrow{(\rho_1, \rho'_1)}_{t_1} p_1w'_1, \dots, p\gamma \xrightarrow{(\rho_h, \rho'_h)}_{t_h} p_hw'_h\}$ is the set of rules in τ with $p\gamma$ as left-hand side, and for $1 \leq i \leq h$, $\sigma_i = (\rho_i, \rho'_i)$, if $t_i \in \{int, ret\}$, then $w_i = w'_i$, and if $t_i = call$ and $w'_i = \gamma_1\gamma_2$, then $w_i = \gamma_1[\gamma_2, p, \gamma]$;
7. if $\phi = EX^a\phi'$, then $[p, EX^a\phi']\gamma \xrightarrow{[(\rho, \rho')]} \{p''w\}$, s.t. either $p\gamma \xrightarrow{(\rho, \rho')}_{int} p'w \in \tau$ and $p'' = [p', \phi']$, or $p\gamma \xrightarrow{(\rho, \rho')}_{call} p'\gamma_1\gamma_2 \in \tau$, $p'' = [p', \phi]^r$, and $w = \gamma_1\gamma_2^r$;
8. if $\phi = EX^a\phi_1$, $\phi = E[\phi_1 U^a \phi_2]$, or $\phi = E[\phi_1 R^a \phi_2]$, then $[p, \phi]^r\gamma \xrightarrow{[(\rho, \rho')]} \{[p', \phi]^r w\} \in \Delta'$ for each $p\gamma \xrightarrow{(\rho, \rho')}_t p'w \in \tau$;
9. if $\phi = AX^a\phi'$, then $[p, AX^a\phi']\gamma \xrightarrow{[\sigma_1, \dots, \sigma_h]} \{p'_1w_1, \dots, p'_hw_h\} \in \Delta'$, s.t. $\{p\gamma \xrightarrow{(\rho_1, \rho'_1)}_{t_1} p_1w'_1, \dots, p\gamma \xrightarrow{(\rho_h, \rho'_h)}_{t_h} p_hw'_h\}$ is the set of rules in τ with $p\gamma$ as left-hand side, and for $1 \leq i \leq h$, $\sigma_i = (\rho_i, \rho'_i)$; if $t_i = int$, then, $p'_i = [p_i, \phi']$ and $w_i = w'_i$; if $t_i = ret$, then $p'_i = True$ and $w_i = w'_i = \varepsilon$; and if $t_i = call$ and $w'_i = \gamma_1\gamma_2$, then $[p_i, \phi']^r$ and $w_i = \gamma_1\gamma_2^r$;
10. if $\phi = AX^a\phi_1$, $\phi = A[\phi_1 U^a \phi_2]$, or $\phi = A[\phi_1 R^a \phi_2]$, then $[p, \phi]^r\gamma \xrightarrow{[\sigma_1, \dots, \sigma_h]} \{[p_1, \phi]^r w_1, \dots, [p_h, \phi]^r w_h\} \in \Delta'$, s.t. $\{p\gamma \xrightarrow{(\rho_1, \rho'_1)}_t p_1w_1, \dots, p\gamma \xrightarrow{(\rho_h, \rho'_h)}_t p_hw_h\}$ is the set of rules in τ with $p\gamma$ as left-hand side, where for $1 \leq i \leq h$, $\sigma_i = (\rho_i, \rho'_i)$;
11. if $\phi = QX^a\phi_1$ for $Q \in \{E, A\}$, then $[p, \phi]^r\gamma^r \xrightarrow{[\top]} \{[p, \phi_1]\gamma\} \in \Delta'$;
12. if $\phi = Q[\phi_1 U^b \phi_2]$ for $Q \in \{E, A\}$ and $b \in \{g, a, c\}$, then $[p, \phi]\gamma \xrightarrow{[\top]} \{[p, \phi_2]\gamma\}$;
13. if $\phi = Q[\phi_1 R^b \phi_2]$ for $Q \in \{E, A\}$ and $b \in \{g, a, c\}$, then $[p, \phi]\gamma \xrightarrow{[\top, \top]} \{[p, \phi_1]\gamma, [p, \phi_2]\gamma\}$;
14. if $\phi = E[\phi_1 U^g \phi_2]$, then $[p, \phi]\gamma \xrightarrow{[(\rho, \rho), (\rho, \rho')]} \{[p, \phi_1]\gamma, [p', \phi]w\}$, s.t. either $p\gamma \xrightarrow{(\rho, \rho')}_{int} p'w \in \tau$, $p\gamma \xrightarrow{(\rho, \rho')}_{ret} p'w \in \tau$, or $p\gamma \xrightarrow{(\rho, \rho')}_{call} p'\gamma_1\gamma_2 \in \tau$ and $w = \gamma_1[\gamma_2, p, \gamma]$;
15. if $\phi = A[\phi_1 U^g \phi_2]$, then $[p, \phi]\gamma \xrightarrow{[\top, \sigma_1, \dots, \sigma_h]} \{[p, \phi_1]\gamma, [p_1, \phi]w_1, \dots, [p_h, \phi]w_h\}$, s.t. $\{p\gamma \xrightarrow{(\rho_1, \rho'_1)}_{t_1} p_1w'_1, \dots, p\gamma \xrightarrow{(\rho_h, \rho'_h)}_{t_h} p_hw'_h\}$ is the set of rules in τ with $p\gamma$ as left-hand side, and for $1 \leq i \leq h$, $\sigma_i = (\rho_i, \rho'_i)$, if $t_i \in \{int, ret\}$, then $w_i = w'_i$, and if $t_i = call$ and $w'_i = \gamma_1\gamma_2$, then $w_i = \gamma_1[\gamma_2, p, \gamma]$;
16. if $\phi = E[\phi_1 U^a \phi_2]$, then $[p, \phi]\gamma \xrightarrow{[(\rho, \rho), (\rho, \rho')]} \{[p, \phi_1]\gamma, p''w\}$, s.t. either $p\gamma \xrightarrow{(\rho, \rho')}_{int} p'w \in \tau$ and $p'' = [p', \phi]$, or $p\gamma \xrightarrow{(\rho, \rho')}_{call} p'\gamma_1\gamma_2 \in \tau$, $p'' = [p', \phi]^r$, and $w = \gamma_1\gamma_2^r$;
17. if $\phi = A[\phi_1 U^a \phi_2]$, then $[p, \phi]\gamma \xrightarrow{[\top, \sigma_1, \dots, \sigma_h]} \{[p, \phi_1]\gamma, p'_1w_1, \dots, p'_hw_h\}$, s.t. $\{p\gamma \xrightarrow{(\rho_1, \rho'_1)}_{t_1} p_1w'_1, \dots, p\gamma \xrightarrow{(\rho_h, \rho'_h)}_{t_h} p_hw'_h\}$ is the set of rules in τ with $p\gamma$

- as left-hand side, and for $1 \leq i \leq h$, $\sigma_i = (\rho_i, \rho'_i)$; if $t_i = \text{int}$, then $p'_i = [p_i, \phi]$ and $w_i = w'_i$; if $t_i = \text{ret}$, then $p'_i = \text{False}$ and $w_i = w'_i = \varepsilon$; and if $t_i = \text{call}$ and $w'_i = \gamma_1 \gamma_2$, then $p'_i = [p_i, \phi]^r$ and $w_i = \gamma_1 \gamma_2^r$;
18. if $\phi = Q[\phi_1 U^a \phi_2]$ or $\phi = Q[\phi_1 R^a \phi_2]$ for $Q \in \{E, A\}$, then $[p, \phi]^r \gamma^r \xrightarrow{[\top]} \{[p, \phi] \gamma\} \in \Delta'$;
 19. if $\phi = E[\phi_1 R^g \phi_2]$, then $[p, \phi] \gamma \xrightarrow{[(\rho, \rho), (\rho, \rho')]} \{[p, \phi_2] \gamma, [p, \phi] w\}$, s.t. either $p\gamma \xrightarrow{(\rho, \rho')}_{\text{int}} p'w \in \tau$, $p\gamma \xrightarrow{(\rho, \rho')}_{\text{ret}} p'w \in \tau$, or $p\gamma \xrightarrow{(\rho, \rho')}_{\text{call}} p'\gamma_1 \gamma_2 \in \tau$ and $w = \gamma_1 [\gamma_2, p, \gamma]$;
 20. if $\phi = A[\phi_1 R^g \phi_2]$, then $[p, \phi] \gamma \xrightarrow{[\top, \sigma_1, \dots, \sigma_h]} \{[p, \phi_2] \gamma, [p_1, \phi] w_1, \dots, [p_h, \phi] w_h\}$, s.t. $\{p\gamma \xrightarrow{(\rho_1, \rho'_1)}_{t_1} p_1 w'_1, \dots, p\gamma \xrightarrow{(\rho_h, \rho'_h)}_{t_h} p_h w'_h\}$ is the set of rules in τ with $p\gamma$ as left-hand side, and for $1 \leq i \leq h$, $\sigma_i = (\rho_i, \rho'_i)$, if $t_i \in \{\text{int}, \text{ret}\}$, then $w_i = w'_i$, and if $t_i = \text{call}$ and $w'_i = \gamma_1 \gamma_2$, then $w_i = \gamma_1 [\gamma_2, p, \gamma]$;
 21. if $\phi = E[\phi_1 R^a \phi_2]$, then $[p, \phi] \gamma \xrightarrow{[(\rho, \rho), (\rho, \rho')]} \{[p, \phi_2] \gamma, p''w\}$, s.t. either $p\gamma \xrightarrow{(\rho, \rho')}_{\text{int}} p'w \in \tau$ and $p'' = [p', \phi]$, or $p\gamma \xrightarrow{(\rho, \rho')}_{\text{ret}} p'w \in \tau$ and $p'' = \text{True}$, or $p\gamma \xrightarrow{(\rho, \rho')}_{\text{call}} p'\gamma_1 \gamma_2 \in \tau$, $p'' = [p', \phi]^r$, and $w = \gamma_1 \gamma_2^r$;
 22. if $\phi = A[\phi_1 R^a \phi_2]$, then $[p, \phi] \gamma \xrightarrow{[\top, \sigma_1, \dots, \sigma_h]} \{[p, \phi_2] \gamma, p'_1 w_1, \dots, p'_h w_h\}$, s.t. $\{p\gamma \xrightarrow{(\rho_1, \rho'_1)}_{t_1} p_1 w'_1, \dots, p\gamma \xrightarrow{(\rho_h, \rho'_h)}_{t_h} p_h w'_h\}$ is the set of rules in τ with $p\gamma$ as left-hand side, and for $1 \leq i \leq h$, $\sigma_i = (\rho_i, \rho'_i)$; if $t_i = \text{int}$, then, $p'_i = [p_i, \phi]$ and $w_i = w'_i$; if $t_i = \text{ret}$, then $p'_i = \text{True}$ and $w_i = w'_i = \varepsilon$; and if $t_i = \text{call}$ and $w'_i = \gamma_1 \gamma_2$, then $p'_i = [p_i, \phi]^r$ and $w_i = \gamma_1 \gamma_2^r$;
 23. if $\phi = QX^c \phi'$, where $Q \in \{E, A\}$, then $[p, \phi] \gamma \xrightarrow{[\top]} \{[p^c, \phi] \varepsilon\}$;
 24. if $\phi = Q[\phi_1 U^c \phi_2]$, where $Q \in \{E, A\}$, then $[p, \phi] \gamma \xrightarrow{[\top, \top]} \{[p, \phi_1] \gamma, [p^c, \phi] \varepsilon\}$;
 25. if $\phi = Q[\phi_1 R^c \phi_2]$, where $Q \in \{E, A\}$, then $[p, \phi] \gamma \xrightarrow{[\top, \top]} \{[p, \phi_2] \gamma, [p^c, \phi] \varepsilon\}$;
 26. $[p^c, \phi] \gamma \xrightarrow{[\top]} \{[p^c, \phi] \varepsilon\} \in \Delta'$ for $\gamma \neq \#$ and $[p^c, \phi] \# \xrightarrow{[\top]} \{[p^c, \phi] \#\}$;
 27. if $\phi = QX^c \phi_1$ for $Q \in \{E, A\}$, $[p^c, \phi] [\gamma, p_0, \gamma_0] \xrightarrow{\text{restore}} [p_0, \phi_1] \gamma_0 \in \text{restore}$ for $p_0 \in P$ and $\gamma_0 \in \Gamma$;
 28. if $\phi = Q[\phi_1 U^c \phi_2]$ or $\phi = Q[\phi_1 R^c \phi_2]$ for $Q \in \{E, A\}$, $[p^c, \phi] [\gamma, p_0, \gamma_0] \xrightarrow{\text{restore}} [p_0, \phi] \gamma_0 \in \text{restore}$ for $p_0 \in P$ and $\gamma_0 \in \Gamma$;
 29. $[p, \phi] [\gamma, p_0, \gamma_0] \xrightarrow{\text{discard}} [p, \phi] \gamma \in \text{discard}$ for $p \neq p^c$, $p_0 \in P$, and $\gamma_0 \in \Gamma$.

Let us explain the intuition behind this construction. Suppose the configuration $(\langle p, \gamma \omega \rangle, \theta)$ of $\mathcal{P}$ satisfies some BranchCARET ϕ . We construct the rules in $\mathcal{BP}$, such that there is an accepting run starting from $(\langle [p, \phi], \gamma \omega \rangle, \theta)$ in $\mathcal{BP}$.

If $\phi = a \in AP$, then item 1 creates a rule of the form $[p, a] \gamma \xrightarrow{[\top]} \{[p, a] \gamma\}$, which creates an infinite loop of the form $(\langle [p, a], \gamma \omega \rangle, \theta) \Rightarrow (\langle [p, a], \gamma \omega \rangle, \theta) \Rightarrow \dots$. And since $[p, a] \in F$, then this infinite loop constitutes an accepting run. The intuition behind $\phi = \neg a$ is similar to this but with item 2 of the construction.

If $\phi = \phi_1 \vee \phi_2$, then either $(\langle p, \gamma \omega \rangle, \theta) \models \phi_1$, or $(\langle p, \gamma \omega \rangle, \theta) \models \phi_2$. Following the idea of the construction inductively, there should be an accepting

run from either $(\langle [p, \phi_1], \gamma\omega \rangle, \theta)$ or $(\langle [p, \phi_2], \gamma\omega \rangle, \theta)$. Item 3 creates rules of the forms $[p, \phi]\gamma \xrightarrow{[\top]} \{[p, \phi_1]\gamma\}$ and $[p, \phi]\gamma \xrightarrow{[\top]} \{[p, \phi_2]\gamma\}$, which create transitions of the forms $(\langle [p, \phi], \gamma\omega \rangle, \theta) \Rightarrow (\langle [p, \phi_1], \gamma\omega \rangle, \theta)$ and $(\langle [p, \phi], \gamma\omega \rangle, \theta) \Rightarrow (\langle [p, \phi_2], \gamma\omega \rangle, \theta)$. Therefore, there is an accepting run from $(\langle [p, \phi], \gamma\omega \rangle, \theta)$ if there is an accepting run from either $(\langle [p, \phi_1], \gamma\omega \rangle, \theta)$, or from $(\langle [p, \phi_2], \gamma\omega \rangle, \theta)$. A similar intuition applies to the case of $\phi = \phi_1 \wedge \phi_2$ and item 4.

If $\phi = EX^g\phi_1$, then there is a global successor $(\langle p', w\omega \rangle, \theta')$ in $\mathcal{P}$ satisfying ϕ_1 and which is obtained from a rule of the form $p\gamma \xrightarrow{(\rho, \rho')}_t p'w \in \tau$, s.t. $\theta' = (\theta \setminus \rho) \cup \rho'$. Item 5 then creates a rule in $\mathcal{BP}$ depending on t (type of the corresponding transition in $\mathcal{P}$):

- If $t = \text{int}$ or $t = \text{ret}$, then Item 5 creates a rule of the form $[p, \phi]\gamma \xrightarrow{[(\rho, \rho')]} \{[p', \phi_1]w\}$. Therefore, in $\mathcal{BP}$ there is a transition of the form $(\langle [p, \phi], \gamma\omega \rangle, \theta) \Rightarrow (\langle [p', \phi_1], w\omega \rangle, \theta')$. Therefore, there is an accepting path in $\mathcal{BP}$ starting from $(\langle [p, \phi], \gamma\omega \rangle, \theta)$ only if there is an accepting path from $(\langle [p', \phi_1], w\omega \rangle, \theta')$, which corresponds to checking if the global successor $(\langle p', w\omega \rangle, \theta')$ satisfies ϕ_1 ;
- If $t = \text{call}$, then the applied rule has the form $p\gamma \xrightarrow{(\rho, \rho')}_t p'\gamma_1\gamma_2 \in \tau$ and $\theta = \theta'$. Item 5 creates a rule of the form $[p, \phi]\gamma \xrightarrow{[(\rho, \rho')]} \{[p', \phi_1]\gamma_1 \upharpoonright \gamma_2, p, \gamma\}$. Therefore, in $\mathcal{BP}$, there is a transition $(\langle [p, \phi], \gamma\omega \rangle, \theta) \Rightarrow (\langle [p', \phi_1], \gamma_1 \upharpoonright \gamma_2, p, \gamma \rangle^\theta \omega, \theta)$.

As explained in Section 3.1, $\upharpoonright \gamma_2, p, \gamma \rangle^\theta$ is the return location γ_2 in $\mathcal{P}$ embedded with the control location p , top of the stack γ , and the phase θ that were just before the *call* happened.

If, in the accepting run of $\mathcal{P}$, the run reaches the corresponding return $c'' \xrightarrow{\text{ret}} c'$, with the return state being of the form $c' = (\langle p'', \gamma_2\omega \rangle, \theta')$ and ϕ requires the return state c' to be checked against some formula ϕ' , the corresponding run in $\mathcal{BP}$ will reach the configuration of the form $(\langle [p'', \phi'], \upharpoonright \gamma_2, p, \gamma \rangle^\theta \omega, \theta')$. Then, the *discard* rule added by Item 29 can be applied, thus obtaining the transition $(\langle [p'', \phi'], \upharpoonright \gamma_2, p, \gamma \rangle^\theta \omega, \theta') \Rightarrow (\langle [p'', \phi'], \gamma_2\omega \rangle, \theta')$.

If, before the corresponding return in $\mathcal{P}$, ϕ requires to check the call site $(\langle p, \gamma\omega \rangle, \theta)$ against some formula ϕ' (via caller modalities), then $\mathcal{BP}$ will use the pushed call history $\upharpoonright \gamma_2, p, \gamma \rangle^\theta$ to restore the call site. The intuition behind this process is explained later alongside the intuition of formulas of the form $EX^c\phi'$.

Therefore, there is an accepting run in $\mathcal{BP}$ starting from $(\langle [p, \phi], \gamma\omega \rangle, \theta)$ only if there is an accepting path from $(\langle [p', \phi_1], \gamma_1 \upharpoonright \gamma_2, p, \gamma \rangle^\theta \omega, \theta')$, which corresponds to checking if the global successor $(\langle p', \gamma_1\gamma_2\omega \rangle, \theta')$ satisfies ϕ_1 ;

The case of $\phi = AX^g\phi_1$ is similar to the case of $EX^g\phi_1$ but using item 6 instead of 5.

If $\phi = EX^a\phi_1$, then we need to check the abstract successor of $(\langle p, \gamma\omega \rangle, \theta)$. The abstract successor depends on the type of rule that can be applied:

- If $\mathcal{P}$ applies a rule of the form $p\gamma \xrightarrow{(\rho, \rho')}_{int} p'w \in \tau$, then the abstract successor is the next configuration $(\langle p', w\omega \rangle, (\theta \setminus \rho) \cup \rho')$. Hence, item 7 constructs a rule of the form $[p, EX^a\phi_1]\gamma \xrightarrow{[(\rho, \rho')]} \{[p', \phi_1]w\}$, which will create transition of the form $(\langle [p, EX^a\phi_1], \gamma\omega \rangle, \theta) \Rightarrow \{(\langle [p', \phi_1], w\omega \rangle, (\theta \setminus \rho) \cup \rho')\}$. Therefore $\mathcal{BP}$ contains an accepting run starting from $(\langle [p, EX^a\phi_1], \gamma\omega \rangle, \theta)$ if there is an accepting run from $(\langle [p', \phi_1], w\omega \rangle, (\theta \setminus \rho) \cup \rho')$.
- If $\mathcal{P}$ applies a rule of the form $p\gamma \xrightarrow{(\rho, \rho')}_{call} p'\gamma_1\gamma_2 \in \tau$, then the abstract successor is the corresponding return location. Suppose the corresponding return in $\mathcal{P}$ is obtained from the path of the form $(\langle p, \gamma\omega \rangle, \theta) \xRightarrow{call} (\langle p', \gamma_1\gamma_2 \rangle, \theta) \Rightarrow^* (\langle p_k, \gamma_k\gamma_2\omega \rangle, \theta') \xRightarrow{ret} \langle p'', \gamma_2\omega \rangle, \theta'$, where $(\langle p'', \gamma_2\omega \rangle, \theta')$ is the corresponding return location. Item 7 constructs a rule of the form $[p, \phi]\gamma \xrightarrow{[(\rho, \rho')]} \{[p', \phi]^r\gamma_1\gamma_2^r\}$, i.e. $\mathcal{BP}$ will have a transition $(\langle [p, \phi], \gamma\omega \rangle, \theta) \Rightarrow \{(\langle [p', \phi]^r, \gamma_1\gamma_2^r\omega \rangle, \theta)\}$. Item 8 ensures that $\mathcal{BP}$ mirrors the execution of $\mathcal{P}$, s.t. $(\langle [p', \phi]^r, \gamma_1\gamma_2^r\omega \rangle, \theta) \Rightarrow^* \{(\langle [p'', \phi]^r, \gamma_2^r\omega \rangle, \theta')\}$. Then, item 11 creates a rule of the form $[p'', \phi]^r\gamma_2^r \xrightarrow{[\top]} \{[p'', \phi_1]\gamma_2\}$, such that $(\langle [p'', \phi]^r, \gamma_2^r\omega \rangle, \theta') \Rightarrow \{(\langle [p'', \phi_1], \gamma_2\omega \rangle, \theta')\}$. Hence, there is an accepting run from $(\langle [p, EX^a\phi_1], \gamma\omega \rangle, \theta)$ in $\mathcal{BP}$ if there is an accepting run from $(\langle [p'', \phi_1], \gamma_2\omega \rangle, \theta')$ in $\mathcal{BP}$, which corresponds to the abstract successor $(\langle p'', \gamma_2\omega \rangle, \theta')$ in $\mathcal{P}$.
- If $\mathcal{P}$ applies a *ret* rule, then there is no abstract successor and $EX^a\phi_1$ is not satisfied. Hence, item 7 does not construct any rules.

If $\phi = AX^a\phi_1$, then $\mathcal{BP}$ must check every possible abstract successor. Suppose $\{p\gamma \xrightarrow{\sigma_1}_{t_1} p_1w_1, \dots, p\gamma \xrightarrow{\sigma_h}_{t_h} p_hw_h\}$ are all rules in τ with $p\gamma$ on the left-hand side. Then, item 9 constructs a rule of the form $[p, AX^a\phi_1]\gamma \xrightarrow{[\sigma_1, \dots, \sigma_h]} \{p'_1w'_1, \dots, p'_hw'_h\}$. For each $1 \leq i \leq h$, p'_i and w'_i depend on the type t_i of the corresponding τ rule:

- If $t_i = int$, then the intuition is similar to the case of $EX^g\phi_1$.
- If $t_i = call$ and $w_i = \gamma_i\gamma'_i$, then $p'_i = [p_i, \phi]^r$ and $w_i = \gamma_i\gamma'^r_i$, i.e. the i -th child of $(\langle [p, AX^a\phi_1], \gamma\omega \rangle, \theta)$ has the form $(\langle [p_i, \phi]^r, \gamma_i\gamma'^r_i\omega \rangle, \theta)$. Then, item 10 will construct such rules that constitute a tree η , where each branch of η starts from $(\langle [p_i, \phi]^r, \gamma_i\gamma'^r_i\omega \rangle, \theta)$ and corresponds to one valid run π in $\mathcal{P}$ until the corresponding return. Let us take one such branch η_π . There are two cases:

- π has a corresponding return $c \xRightarrow{ret} (\langle p'', \gamma'_2\omega \rangle, \theta')$ for some $c \in Conf$.

Then, rules created by item 10 ensure that η_π will reach $(\langle [p'', \phi]^r, \gamma'_2\omega \rangle, \theta')$, and rules constructed by item 11 allow the transition $(\langle [p'', \phi]^r, \gamma'_2\omega \rangle, \theta') \Rightarrow (\langle [p'', \phi], \gamma'_2\omega \rangle, \theta')$. Hence, η_π is accepting only if $\mathcal{BP}$ contains an accepting run from $(\langle [p'', \phi_1], \gamma'_2\omega \rangle, \theta')$, which corresponds to the abstract successor $(\langle p'', \gamma'_2\omega \rangle, \theta')$ in $\mathcal{P}$;

- π does not have a corresponding return, i.e. it contains an infinite loop before reaching a corresponding return. Then, η_π will never contain γ_2^r on top of the stack. By the construction of $\mathcal{BP}$, the control locations of the form $[p, \phi]^r$ can ‘remove’ the mark r only using the rule created by Item 11. Hence, every control location on η_π has the form $[p_k, \phi]^r$ for any $p_k \in P$. Since $[p_k, AX^a \phi_1]^r \in F^a \subseteq F$, then η_π is accepting.
- If $t_i = \text{ret}$, then there is no abstract successor, and $p'_i = \text{True}$, i.e. the i -th child has the form $(\langle \text{True}, \omega \rangle, \theta)$, from which there is always an accepting run. Such branch is required to cover the case when there are only *ret* rules that can be applied. In this case, there are no abstract successors, thus, it satisfies $AX^a \phi_1$, and therefore, this run needs to be accepting.

Therefore, there is an accepting run from $(\langle [p, AX^a \phi_1], \gamma\omega \rangle, \theta)$ if for each abstract successor $(\langle p_i, w_i\omega \rangle, \theta_i)$ in $\mathcal{P}$, there is an accepting run in $\mathcal{BP}$ starting from $(\langle [p_i, \phi_1], w_i\omega \rangle, \theta_i)$.

If $\phi = E[\phi_1 U^g \phi_2]$. Suppose that $\pi(i) = (\langle p, \gamma\omega \rangle, \theta)$ and $(\pi, i) \models \phi$. Then either $(\pi, i) \models \phi_2$, or $(\pi, i+1) \models E[\phi_1 U^g \phi_2]$ and $(\pi, i) \models \phi_1$. In the first case, item 12 creates a rule of the form $[p, \phi]\gamma \xrightarrow{[\top]} \{[p, \phi_2]\gamma\}$ and therefore, there is an accepting run from $(\langle [p, \phi], \gamma\omega \rangle, \theta)$ if there is an accepting run from $(\langle [p, \phi_2], \gamma\omega \rangle, \theta)$.

If $(\pi, i) \models \phi_1$ and $(\pi, i+1) \models E[\phi_1 U^g \phi_2]$, then let $p\gamma \xrightarrow{(\rho, \rho')}_{\text{int}} p'w$ be the rule applied at (π, i) . Item 14 creates a rule of the form $[p, \phi]\gamma \xrightarrow{[(\rho, \rho), (\rho, \rho')]} \{[p, \phi_1]\gamma, [p', \phi]w\}$, i.e. there is an accepting run from $(\langle [p, \phi], \gamma\omega \rangle, \theta)$ if there are accepting runs from both $(\langle [p, \phi_1], \gamma\omega \rangle, \theta)$ and $(\langle [p', \phi], w\omega \rangle, (\theta \setminus \rho) \cup \rho')$. If the rule applied at (π, i) has the form $p\gamma \xrightarrow{(\rho, \rho')}_{\text{call}} p'\gamma_1\gamma_2$, then item 14 constructs a rule of the form $[p, \phi]\gamma \xrightarrow{[(\rho, \rho), (\rho, \rho')]} \{[p, \phi_1]\gamma, [p', \phi]\gamma_1[\gamma_2, p, \gamma]\}$, i.e. there is an accepting run from $(\langle [p, \phi], \gamma\omega \rangle, \theta)$ if there are accepting runs from both $(\langle [p, \phi_1], \gamma\omega \rangle, \theta)$ and $(\langle [p', \phi], \gamma_1[\gamma_2, p, \gamma]\omega \rangle, \theta)$. The intuition behind the checkpoint $[\gamma_2, p, \gamma]$ is explained in Section 3.1 as well as in the cases of $EX^g \phi_1$ and $EX^c \phi_1$. Therefore, there is an accepting run from $(\langle [p, \phi], \gamma\omega \rangle, \theta)$ if $(\langle p, \gamma\omega \rangle, \theta) \models \phi_1$ and there is a successor $(\langle p', w\omega \rangle, \theta')$, s.t. $(\langle p', w\omega \rangle, \theta') \models \phi$. Note that $\mathcal{BP}$ cannot promise indefinitely that the successor satisfies ϕ , because at some point, ϕ_2 must be satisfied. We ensure this by the definition of F (i.e. $[p, E[\phi_1 U^g \phi_2]] \notin F$), such that a run infinitely promising $E[\phi_1 U^g \phi_2]$ is possible, but it will not be accepting.

If $\phi = A[\phi_1 U^g \phi_2]$, the intuition is similar to $\phi = E[\phi_1 U^g \phi_2]$.

If $\phi = E[\phi_1 U^a \phi_2]$. Suppose that $\pi(i) = (\langle p, \gamma\omega \rangle, \theta)$ and $(\pi, i) \models \phi$. Then either $(\pi, i) \models \phi_2$, or $(\pi, \text{next}_\pi^a(i)) \models E[\phi_1 U^a \phi_2]$ and $(\pi, i) \models \phi_1$. Intuition behind checking $(\pi, i) \models \phi_2$ is the same as in $\phi = E[\phi_1 U^g \phi_2]$. If $(\pi, \text{next}_\pi^a(i)) \models E[\phi_1 U^a \phi_2]$ and $(\pi, i) \models \phi_1$, item 16 constructs a rule depending on the type of rule that was applied at (π, i) :

- If the rule has the form $p\gamma \xrightarrow{(\rho, \rho')}_{\text{int}} p'w$, then the intuition is the same as in $E[\phi_1 U^g \phi_2]$;
- If the rule has the form $p\gamma \xrightarrow{(\rho, \rho')}_{\text{call}} p'\gamma_1\gamma_2$, then item 16 constructs a rule of the form $[p, \phi]\gamma \xrightarrow{[(\rho, \rho), (\rho, \rho')]} \{[p, \phi_1]\gamma, [p', \phi]^r\gamma_1\gamma_2^r\}$, thus allowing the tran-

sition $(\langle [p, \phi], \gamma\omega \rangle, \theta) \implies \{(\langle [p, \phi_1], \gamma\omega \rangle, \theta), (\langle [p', \phi]^r, \gamma_1\gamma_2^r\omega \rangle, \theta)\}$. There are two cases depending on whether the abstract successor of $\pi(i)$ exists:

- Suppose the abstract successor of $\pi(i)$ exists and $\pi(\text{next}_\pi^a(i)) = (\langle p'', \gamma_2\omega \rangle, \theta')$. Similar to the case of $EX^a\phi_1$, $(\langle [p', \phi]^r, \gamma_1\gamma_2^r\omega \rangle, \theta)$ will reach the configuration $(\langle [p'', \phi], \gamma_2 \rangle, \theta')$, which corresponds to the abstract successor $\pi(\text{next}_\pi^a(i))$. Hence there is an accepting run in $\mathcal{BP}$ starting from $(\langle [p, \phi], \gamma\omega \rangle, \theta)$ if there are accepting runs from both $(\langle [p, \phi_1], \gamma\omega \rangle, \theta)$ and $(\langle [p'', \phi], \gamma_2\omega \rangle, \theta')$, i.e. if $(\pi, \text{next}_\pi^a(i)) \models \phi$ and $(\pi, i) \models \phi_1$. Moreover, the condition that ϕ_2 is satisfied in the future is guaranteed by the fact that $[p, E[\phi_1 U^a \phi_2]] \notin F$ for any $p \in P$.
 - If the abstract successor $\text{next}_\pi^a(i)$ is undefined, i.e. there is no corresponding return. Unlike with $AX^a\phi_1$, (π, i) should not satisfy ϕ , because ϕ_2 is never satisfied on the abstract path on π . In this case, $\mathcal{BP}$ will visit only control locations of the form $[p_k, \phi]^r$ after $\pi(i)$. This run is not accepting because $[p_k, \phi]^r \notin F$.
- If the rule has the form $p\gamma \xrightarrow{\sigma}_{ret} p'w$, then there is no abstract successor, so item 16 does not construct the corresponding rule and hence, there is no accepting run from $(\langle [p, \phi], \gamma\omega \rangle, \theta)$.

If $\phi = A[\phi_1 U^a \phi_2]$. The intuition behind this case is similar to $\phi = E[\phi_1 U^a \phi_2]$ but with construction item 17 instead of 16. However, there is one case in which the semantics of $A[\phi_1 U^a \phi_2]$ is different. Suppose there is a run $\pi' \in \text{Ext}(\pi, i)$, s.t. at $\pi'(i)$ a return rule of the form $p\gamma \xrightarrow{\sigma}_{ret} p'$ is applied. That is, there is no abstract successor of $\pi'(i)$, i.e. $\text{next}_{\pi'}^a(i) = \perp$. Unlike with $AX^a\phi_1$, the absence of the abstract successor on π' does not imply the absence of the abstract path in π' . In fact, the abstract path starting from $\pi'(i)$ is a sequence $s = i$, which contains a single element i . Therefore, ϕ_2 must be satisfied on s . However, since s is a single element i , then (π', i) must satisfy ϕ_2 , i.e. (π, i) must satisfy ϕ_2 . In this case, the rule added by Item 17 should not be applied. To ensure this, the control location in $\mathcal{BP}$ that corresponds to the application of rule of the form $p\gamma \xrightarrow{\sigma}_{ret} p'$ is *False*, i.e. $\mathcal{BP}$ always rejects runs that can perform *return* and try to check the abstract successors. Instead, the accepting run must be found by applying a rule from Item 12, which corresponds to checking if $(\pi, i) \models \phi_2$.

If $\phi = E[\phi_1 R^g \phi_2]$. Suppose that $\pi(i) = (\langle p, \gamma\omega \rangle, \theta)$ and $(\pi, i) \models \phi$. There are two cases: either there is $k \geq i$, s.t. $(\pi, k) \models \phi_1$, and for each $i \leq j \leq k$ $(\pi, j) \models \phi_2$, or for all $k \geq i$, $(\pi, k) \models \phi_2$. In the first case, the intuition is similar to the case of $E[\phi_1 U^g \phi_2]$. In the latter case, item 19 allows to construct a run η in $\mathcal{BP}$ starting from $(\langle [p, \phi], \gamma\omega \rangle, \theta)$, such that there is an infinite branch η_π corresponding to the run π in $\mathcal{P}$, where if $\pi(j) = (\langle p_j, w_j \rangle, \theta_j)$, then the j -th node of η_π has the form $(\langle [p_j, \phi], w_j \rangle, \theta_j)$ and has two children: $(\langle [p_{j+1}, \phi], w_{j+1} \rangle, \theta_{j+1})$ and $(\langle [p_j, \phi_2], w_j \rangle, \theta_j)$. Hence, there is an accepting run in $\mathcal{BP}$ starting from $(\langle [p, \phi], \gamma\omega \rangle, \theta)$ if there are accepting runs from $(\langle [p_j, \phi_2], w_j \rangle, \theta_j)$ for each $j \geq i$ and η_π is an accepting branch. The latter is derived from the definition of F , as $[p, E[\phi_1 R^g \phi_2]] \in F$ for any $p \in P'$.

If $\phi = A[\phi_1 R^g \phi_2]$, then the intuition is similar to $\phi = E[\phi_1 R^g \phi_2]$.

If $\phi = E[\phi_1 R^a \phi_2]$. Suppose that $\pi(i) = (\langle p, \gamma\omega \rangle, \theta)$ and $(\pi, i) \models \phi$. Then let $s = s_0 s_1 \dots$ be a sequence, s.t. $s_0 = i$ and for $0 \leq j$, $s_{j+1} = \text{next}_\pi^a(s_j)$. $(\pi, i) \models \phi$ if either there is k , s.t. $(\pi, s_k) \models \phi_1$ and for each $j \leq k$, $(\pi, s_j) \models \phi_2$, or $(\pi, s_j) \models \phi_2$ for each $j \geq 0$. The first case is similar to $E[\phi_1 U^a \phi_2]$. In the second case, there are two possibilities:

- s is finite. Suppose $s = s_0 \dots s_k$. Item 21 allows to construct a tree with the root $(\langle [p, \phi], \gamma\omega \rangle, \theta)$ and a finite branch η_π of length s_k that corresponds to the run π in $\mathcal{P}$. The l -th node on this branch can take one of the two forms - if $l = s_j$ for some j , then $(\langle [p_l, \phi], w_l \rangle, \theta_l)$, and else $(\langle [p_l, \phi]^r, w_l \rangle, \theta_l)$ (following the same intuition as in $E[\phi_1 U^a \phi_2]$). Suppose $w_l = \gamma_l \omega_l$. According to item 21, each node of the form $(\langle [p_l, \phi], w_l \rangle, \theta_l)$ has one child of the form $(\langle [p_l, \phi_2], w_l \rangle, \theta_l)$ and another child on the branch η_π . Then, there is an accepting run from $(\langle [p, \phi], \gamma\omega \rangle, \theta)$ if there are accepting runs from each $(\langle [p_{s_j}, \phi_2], w_{s_j} \rangle, \theta_{s_j})$ and from $(\langle [p_{s_k}, \phi], w_{s_k} \rangle, \theta_{s_k})$. There are two cases for the reason why s is finite:
 - The rule applied at step s_k is a *ret* rule of the form $p_{s_k} \gamma_{s_k} \xrightarrow{\sigma}_{ret} p''$. Then item 21 creates a rule of the form $[p_{s_k}, \phi] \gamma_{s_k} \xrightarrow{[\sigma, \sigma]} \{[p_{s_k}, \phi_2] \gamma_{s_k}, \text{True}\}$. Thus, there is a transition in $\mathcal{BP}$ of the form $(\langle [p_{s_k}, \phi], w_{s_k} \rangle, \theta_{s_k}) \Rightarrow \{(\langle [p_{s_k}, \phi_2], w_{s_k} \rangle, \theta_{s_k}), (\langle \text{True}, \omega_{s_k} \rangle, \theta')\}$. There is always an accepting run from $(\langle \text{True}, \omega' \rangle, \theta')$, and $(\langle [p_{s_k}, \phi_2], w_{s_k} \rangle, \theta_{s_k})$ is a child of the node on η_π . Therefore, there is an accepting run from $(\langle [p, \phi], \gamma\omega \rangle, \theta)$ if each configuration of the form $(\langle [p_{s_j}, \phi], w_{s_j} \rangle, \theta_{s_j})$ on the abstract path satisfies ϕ_2 .
 - The rule applied at step s_k is a call rule of the form $p_{s_k} \gamma_{s_k} \xrightarrow{\sigma}_{call} p'' \gamma' \gamma''$ without the corresponding return. Then, item 21 creates a rule of the following form: $[p_{s_k}, \phi] \gamma_{s_k} \xrightarrow{[\sigma, \sigma]} \{[p_{s_k}, \phi_2] \gamma_{s_k}, [p'', \phi]^r \gamma' \gamma''^r\}$. This rule allows $(\langle [p_{s_k}, \phi], w_{s_k} \rangle, \theta_{s_k}) \Rightarrow \{(\langle [p_{s_k}, \phi_2], w_{s_k} \rangle, \theta_{s_k}), (\langle [p'', \phi]^r, \gamma' \gamma''^r \omega_{s_k} \rangle, \theta_{s_k})\}$. And since there is no corresponding return from $(\langle p'', \gamma' \gamma'' \omega_{s_k} \rangle, \theta_{s_k})$, then Item 8 creates rules that constitute an infinite run through control locations of the form $[p, \phi]^r$. And this run is accepting because such control locations are in $F^a \subseteq F$. Therefore, there is an accepting run from $(\langle [p, \phi], \gamma\omega \rangle, \theta)$ if each configuration of the form $(\langle [p_{s_j}, \phi], w_{s_j} \rangle, \theta_{s_j})$ on the abstract path satisfies ϕ_2 .
- s is infinite. Then, the intuition is similar to the intuition of $E[\phi_1 R^g \phi_2]$.

If $\phi = A[\phi_1 R^a \phi_2]$, then the intuition is similar to the case of $\phi = E[\phi_1 R^a \phi_2]$ with item 22.

If $\phi = EX^c \phi_1$. The intuition behind this case follows the explanation in Section 3.1. Suppose $\pi(i) = (\langle p, w \rangle, \theta)$ $(\pi, i) \models EX^c \phi_1$. Then, there is a caller successor $\text{next}_\pi^c(i) = k$, s.t. $(\pi, k) \models \phi_1$. Since k is a caller successor, then let $\pi(k) = (\langle p_k, \gamma_k \omega \rangle, \theta_k)$ and the call rule be of the form $p_k \gamma_k \xrightarrow{\sigma}_{call} p_{k+1} \gamma' \gamma''$. Thus, the transition in π would be of the form $(\langle p_k, \gamma_k \omega \rangle, \theta_k) \Rightarrow (\langle p_{k+1}, \gamma' \gamma'' \omega \rangle, \theta_k)$. Then $\pi(i)$ would be of the form $(\langle p, w' \gamma'' \omega \rangle, \theta)$, where γ'' is the corresponding return location. According to the construction of $\mathcal{BP}$, we encode γ'' with

the call history, thus obtaining that the current configuration in $\mathcal{BP}$ is of the form $(\langle [p, \phi], w'[\gamma'', p_k, \gamma_k]^{\theta_k \omega} \rangle, \theta)$. By Item 23, if $w' = \gamma_i w''$, then $\mathcal{BP}$ allows the transition $(\langle [p, \phi], w'[\gamma'', p_k, \gamma_k]^{\theta_k \omega} \rangle, \theta) \Longrightarrow (\langle [p^c, \phi], w''[\gamma'', p_k, \gamma_k]^{\theta_k \omega} \rangle, \theta)$.

Then, rule created by Item 26 pops w'' from the stack in the following fashion: $(\langle [p^c, \phi], w''[\gamma'', p_k, \gamma_k]^{\theta_k \omega} \rangle, \theta) \Longrightarrow (\langle [p^c, \phi], [\gamma'', p_k, \gamma_k]^{\theta_k \omega} \rangle, \theta)$. Then, item

27 creates a rule of the form $[p^c, \phi][\gamma'', p_k, \gamma_k] \xrightarrow{\text{restore}} [p_k, \phi_1]\gamma_k$. Therefore, there is transition $(\langle [p^c, \phi], [\gamma'', p_k, \gamma_k]^{\theta_k \omega} \rangle, \theta) \Longrightarrow (\langle [p_k, \phi_1], \gamma_k \omega \rangle, \theta_k)$. There-

fore, there is an accepting run from $(\langle [p, EX^c \phi_1], w'[\gamma'', p_k, \gamma_k]^{\theta_k \omega} \rangle, \theta)$ if the caller successor $(\langle p_k, \gamma_k \omega \rangle, \theta_k)$ satisfies ϕ_1 .

If $\phi = E[\phi_1 U^c \phi_2]$. Suppose that $\pi(i) = (\langle p, \gamma \omega \rangle, \theta)$ and $(\pi, i) \models \phi$. Then either $(\pi, i) \models \phi_2$, or $(\pi, i) \models \phi_1$ and $(\pi, next_\pi^c(i)) \models \phi$. The first case is similar to $E[\phi_1 U^g \phi_2]$. In the second case, item 24 constructs a rule of the form

$[p, E[\phi_1 U^c \phi_2]]\gamma \xrightarrow{[\top, \top]} \{[p, \phi_1]\gamma, [p^c, E[\phi_1 U^c \phi_2]]\varepsilon\}$. Similar to the case of $EX^c \phi$, there is a transition $(\langle [p^c, \phi], \omega \rangle, \theta) \Longrightarrow^* \{(\langle [p', \phi], w' \rangle, \theta')\}$, where $\pi(next_\pi^c(i)) =$

$(\langle p', w' \rangle, \theta')$. Therefore, there is an accepting run from $(\langle [p, \phi], \gamma \omega \rangle, \theta)$ if there are accepting runs from both $(\langle [p, \phi_1], \gamma \omega \rangle, \theta)$ and $(\langle [p', \phi], w' \rangle, \theta')$, i.e. if $(\langle p, \gamma \omega \rangle, \theta)$ satisfies ϕ_1 and the caller successor $(\langle p', w' \rangle, \theta')$ satisfies ϕ . The condition that ϕ_2 must be satisfied on the caller path is ensured by the definition of F . Suppose for some k , $next_\pi^k(k) = \perp$ and $\pi(k) = (\langle p_k, w_k \# \rangle, \theta_k)$. In this case, the corresponding configuration in $\mathcal{BP}$ is of the form $(\langle [p_k, \phi], w_k \# \rangle, \theta_k)$, s.t. there are no checkpoints in w_k (because there are no prior calls that pushed a checkpoint). The rule obtained from Item 26 will pop w_k from the stack, obtaining the configuration $(\langle [p^c, \phi], \# \rangle, \theta_k)$. Then, a rule obtained from Item 26 will create an infinite loop over $(\langle [p^c, \phi], \# \rangle, \theta_k)$. And since $[p^c, \phi] \notin F$, then this run is not accepting. The same intuition is applied for $A[\phi_1 U^c \phi_2]$ because (π, i) can have at most one caller successor.

If $\phi = E[\phi_1 R^c \phi_2]$. Suppose that $\pi(i) = (\langle p, \gamma \omega \rangle, \theta)$ and $(\pi, i) \models \phi$. Then let $s = s_0 s_1 \dots s_k$ be the sequence such that $s_0 = i$, and for $0 \leq j < k$, $s_{j+1} = next_\pi^c(s_j)$ and $next_\pi^c(s_k) = \perp$. There are two cases: either there is $0 \leq l \leq k$, s.t. $(\pi, s_l) \models \phi_1$ and for each $0 \leq j \leq l$, $(\pi, s_j) \models \phi_2$, or for each $0 \leq j \leq k$, $(\pi, s_j) \models \phi_2$. The intuition behind the first case is similar to $E[\phi_1 U^c \phi_2]$. In the second case, following the same logic as with $\phi = E[\phi_1 R^a \phi_2]$, if $\pi(s_k) = (\langle p_{s_k}, w_{s_k} \rangle, \theta_{s_k})$, then item 25 creates rules such that there is a tree η starting from $(\langle [p, \phi], \gamma \omega \rangle, \theta)$ with a branch η^c containing nodes of either form $(\langle [p^c, \phi], w' \rangle, \theta')$ or $(\langle [p_{s_j}, \phi], w_{s_j} \rangle, \theta_{s_j})$ for $0 \leq j \leq k$. Then, for $0 \leq j < k$, the node $(\langle [p_{s_j}, \phi], w_{s_j} \rangle, \theta_{s_j})$ has one child of the form $(\langle [p_{s_j}, \phi_2], w_{s_j} \rangle, \theta_{s_j})$ and another node of the form $(\langle [p^c, \phi], w' \rangle, \theta')$ on η^c . Then, at s_k , $\mathcal{BP}$ can apply a rule of the form $[p^c, \phi]\gamma \xrightarrow{[\top, \top]} \{[p, \phi_2]\gamma, [p^c, \phi]\varepsilon\}$. Then, since $next_\pi^c(s_k) = \perp$, there is an accepting run from $(\langle [p^c, \phi], w_{s_k} \# \rangle, \theta_{s_k})$ created by rules from Item 26 that pop w_{s_k} from the stack and create an infinite loop over $(\langle [p^c, \phi], \# \rangle, \theta_{s_k})$. And since $[p^c, \phi] \in F^c \subseteq F$, then this run is accepting. A similar intuition is applied for the cases $\phi = AX^c \phi_1$ and $\phi = A[\phi_1 R^c \phi_2]$.

Thus, we can show that (the proof can be found in Appendix A):

Theorem 1. *For a configuration $(\langle p, w \rangle, \theta)$ of $\mathcal{P}$ and a BranchCARET formula ψ , $(\langle p, w \rangle, \theta) \models \psi$ iff there is an accepting run in $\mathcal{BP}$ starting from $(\langle [p, \psi], w \rangle, \theta)$.*

3.3 Solving the Emptiness Problem of SM-ABPDS with Checkpoints

Following [23], to find accepting runs in $\mathcal{BP}$, we need to define functions $pre^+, pre^* : 2^{Conf_{\mathcal{BP}}} \rightarrow 2^{Conf_{\mathcal{BP}}}$, which compute the transitive and reflexive-transitive sets of predecessors respectively. The functions are defined as $pre^*(W) = \{c \mid c \in W \text{ or } c \implies S, \text{ s.t. } S \subseteq pre^*(W)\}$, and $pre^+(W) = \{c \mid c \implies S, \text{ s.t. } S \subseteq pre^*(W)\}$. Let $Y_0 = Conf_{\mathcal{BP}}$ be the set of configurations from which the accepting control locations are visited at least 0 times. Then, for $j \leq 0$, let $Y_{j+1} = pre^+(Y_j \cap (F \times G^* \times 2^\delta))$ be the set of configurations from which the accepting control locations are visited at least j times. Then, we can find the set of configurations $Y_{\mathcal{BP}}$, from which accepting control locations are visited infinitely many times by defining $Y_{\mathcal{BP}} = \bigcap_{j=0} Y_j$. We can show that (the proof can be found in Appendix B):

Theorem 2. *For any configuration c of $\mathcal{BP}$, there is an accepting run starting from c iff $c \in Y_{\mathcal{BP}}$.*

Intuitively, suppose there is an accepting run η from c in $\mathcal{BP}$. By definition, $c \in Y_0$. Then, each branch of η must visit an accepting control location at least once. Suppose for the i -th branch, it visits the first accepting control location at node c_i , i.e. $c_i \in F \times G^* \times 2^\delta$, by definition of Y_0 , $c_i^1 \in Y_0$ and hence, $c_i \in Y_0 \cap F \times G^* \times 2^\delta$. Moreover, since c reaches $\bigcup_i c_i$ in η , then $c \in pre^+(\bigcup_i c_i)$, and thus, $c \in Y_1$. Since each branch of η must visit accepting control locations at least twice, then for each c_i , there must be a run starting from c_i that visits accepting control locations at least once. Inductively, we get that $c_i \in Y_1$. And therefore, $c \in Y_2$ (because $c_i \in F \times G^* \times 2^\delta$ and $c \in pre^+(\bigcup_i c_i)$). Then, since each branch of η must visit accepting control locations infinitely many times, we can inductively apply this logic for each $j \geq 0$, obtaining that $c \in Y_j$. And thus, since $Y_{\mathcal{BP}} = \bigcap_{j \geq 0} Y_j$, we get that $c \in Y_{\mathcal{BP}}$.

3.4 Computing $Y_{\mathcal{BP}}$

We show that we can efficiently compute $Y_{\mathcal{BP}}$ by constructing a kind of finite automaton $\mathcal{A}$.

Definition 6. *An Alternating Multi Automaton (AMA) is a tuple $A = (Q, G, T, I, F')$, where Q is a finite set of states, G is the alphabet, T is the set of transitions of the form $q \xrightarrow{\gamma} S$, where $q \in Q$, $S \subseteq Q$, and $\gamma \in G$, $I \subseteq Q$ is the set of initial states, and $F' \subseteq Q$ is the set of accepting states.*

We define the reflexive-transitive closure of the $\rightarrow$ relation as follows: $q \xrightarrow{\varepsilon}^* q$, and $q \xrightarrow{\gamma\omega}^* S$ iff $q \xrightarrow{\gamma} S'$ for some $S' \subseteq Q$, $\gamma \in G$, $\omega \in G^*$, and for each $q' \in S'$, $q' \xrightarrow{\omega}^* S_{q'}$, and $S = \bigcup_{q' \in S'} S_{q'}$.

To compute $Y_{\mathcal{BP}}$, we construct an AMA $\mathcal{A} = (Q, G, T, I, F')$, such that $I \subseteq P' \times 2^\delta \times \mathbb{N}$ is the set of enumerated states of the form $(p, \theta)^i$, where $p \in P'$ is a control location in $\mathcal{BP}$, $\theta \subseteq \delta$ is a phase in $\mathcal{BP}$, $Q = I \cup \{q_f\}$, and $F' = \{q_f\}$, where q_f is a single accepting state.

We show that we can construct $\mathcal{A}$ using Algorithm 1. To force the termination of the algorithm, we define the acceleration function pr for some set of states $S \in Q$ of $\mathcal{A}$, defined as:

$$pr^{-1}(S) = \begin{cases} \{q^k \mid q^{k+1} \in S\} \cup \{q_f\} & \text{if } q_f \in S \text{ or } \exists q^1 \in S \\ \{q^k \mid q^{k+1} \in S\} & \text{else} \end{cases}$$

$$pr^k(S) = \{q^k \mid \exists j, 1 \leq j \leq k : q^j \in S\} \cup \{q_f \mid q_f \in S\}$$

The $pr^{-1}(S)$ function projects states in S of the form q^k onto states of the form q^{k-1} , such that rules of the form q^1 are projected onto q_f . The $pr^k(S)$ function projects states of the form q^j in S for any j onto states of the form q^k without changing q_f . We can show that (the proof can be found in Appendix C):

Algorithm 1 Computation of $Y_{\mathcal{BP}_i}$

Input: An SM-ABPDS $\mathcal{BP}_i = (P', \Gamma', X, \Delta', \text{restore}, \text{discard}, F, \delta)$.
Output: An AMA $\mathcal{A}_i = (Q, G, T, I, \{q_f\})$.

- 1: $k := 0$, $T := \{q_f \xrightarrow{\gamma} \{q_f\} \mid \forall \gamma \in \Gamma', \forall p \in P'_i \text{ and } \forall \theta \subseteq \delta, (p, \theta)^0 := q_f\}$;
- 2: **repeat**(we call this *loop*₁)
- 3: $k := k + 1$;
- 4: For all $p \in F, \theta \subseteq \delta$, add $(p, \theta)^k \xrightarrow{\varepsilon} \{(p, \theta)^{k-1}\} \in T$;
- 5: **repeat**(we call this *loop*₂)
- 6: **for all** $\theta \subseteq \delta, p\gamma \xrightarrow{[\sigma_1, \dots, \sigma_h]} \{p_1 w_1, \dots, p_h w_h\} \in \tau$ **do**
- 7: let $\sigma_j = (\rho_j, \rho'_j)$ and $\theta_j = (\theta \setminus \rho_j) \cup \rho'_j$;
- 8: let $W_j = \{R_j \mid \rho_j \subseteq \theta, (p_j, \theta_j)^k \xrightarrow{em(\theta, w_j)}^* R_j\}$;
- 9: $T := T \cup \{(p, \theta)^k \xrightarrow{\gamma} \bigcup_{R \in W} R \mid W \in \prod_{j=1}^h W_j\}$;
- 10: **end for**
- 11: **for all** $p\chi \hookrightarrow p'\gamma \in \text{restore}$, s.t. $\exists(p', \theta) \xrightarrow{\gamma} R$ **do**
- 12: $T := T \cup \{(p, \theta')^k \xrightarrow{\chi^\theta} R \mid \theta' \subseteq \delta\}$;
- 13: **end for**
- 14: **for all** $p\chi \hookrightarrow p'\gamma \in \text{discard}$, s.t. $\exists(p', \theta) \xrightarrow{\gamma} R$ **do**
- 15: $T := T \cup \{(p, \theta)^k \xrightarrow{\chi^{\theta'}} R \mid \theta' \subseteq \delta\}$;
- 16: **end for**
- 17: **until** No new transitions can be added;
- 18: For all $p \in F, \theta \subseteq \delta$, remove $(p, \theta)^k \xrightarrow{\varepsilon} \{(p, \theta)^{k-1}\}$ from T ;
- 19: For all $p \in P', \theta \subseteq \delta, \gamma \in G, R \subseteq Q$, replace $(p, \theta)^k \xrightarrow{\gamma} R$ by $(p, \theta)^k \xrightarrow{\gamma} pr^k(R)$;
- 20: **until** $k > 1$ and $\forall p \in P', \gamma \in G, \theta \subseteq \delta, R \subseteq P' \times 2^\delta \times \{k\} \cup \{q_f\}, (p, \theta)^k \xrightarrow{\gamma} R \in T \iff (p, \theta)^{k-1} \xrightarrow{\gamma} pr^{-1}(R) \in T$;

Theorem 3. *Algorithm 1 terminates and computes an AMA $\mathcal{A}$, s.t. there is a path in $\mathcal{A}$ of the form $(p, \theta)^k \xrightarrow{w} \{q_f\}$ iff $(\langle p, w \rangle, \theta) \in Y_{\mathcal{BP}}$.*

Construction of $\mathcal{A}$ follows the computation steps of $Y_{\mathcal{BP}} = \bigcap_{i \geq 0} Y_i$. Intuitively, $\mathcal{A}$ is a kind of a collection of AMAs A_i (that contain states of the form q^i) for $i \geq 0$, such that a configuration $(\langle p, \omega \rangle, \theta) \in Y_i$ iff $(p, \theta)^i \xrightarrow{\omega}^* \{q_f\}$ in $\mathcal{A}$. Let the set of configurations $(\langle p, \omega \rangle, \theta)$, such that $(p, \theta)^i \xrightarrow{\omega}^* \{q_f\}$ in $\mathcal{A}$ at a given point of execution be denoted as $L(A_i)$. The algorithm consists of two loops: *loop*₁ and *loop*₂. After the i -th iteration of *loop*₁, $\mathcal{A}$ contains states of the form q^i , such that there is a path $(p, \theta)^i \xrightarrow{w}^* \{q_f\}$ iff $(\langle p, w \rangle, \theta) \in Y_i$. This is ensured as follows. First, since $Y_0 = P' \times G^* \times \delta$, Line 1 creates transitions such that $Y_0 = L(A_0)$. Then, for each iteration i of *loop*₁, Line 4 adds such transitions that $L(A_i) = L(A_{i-1}) \cap F \times G^* \times 2^\delta$, which corresponds to $Y_{i-1} \cap F \times G^* \times 2^\delta$. Then, *loop*₂ computes the set of predecessors pre^* , such that after *loop*₁, $L(A_i) = pre^*(L(A_{i-1}) \cap F \times G^* \times 2^\delta)$, which corresponds to $pre^*(Y_{i-1} \cap F \times G^* \times 2^\delta)$. Then, Line 18 removes the initial rules added on Line 4. Since these rules were added before *loop*₂, we obtain that $L(A_i) = pre^+(L(A_{i-1}) \cap F \times G^* \times 2^\delta)$, which corresponds to $Y_i = pre^+(Y_{i-1} \cap F \times G^* \times 2^\delta)$.

Therefore, from Theorems 1, 2, and 3, we can show that:

Theorem 4. *We can construct an AMA $\mathcal{A}$, such that for any configuration $(\langle p, w \rangle, \theta)$ of $\mathcal{P}$ and any BranchCARET formula ψ , $(\langle p, w \rangle, \theta) \models \psi$ iff $([p, \psi], \theta) \xrightarrow{w}^* \{q_f\}$ in $\mathcal{A}$.*

3.5 Model Checking with Regular Valuations

Sometimes it is required to reason about stack contents in the BranchCARET atomic propositions. We can describe the stack using regular languages:

Definition 7. *For an SM-PDS $\mathcal{P} = (P, \Gamma, \Delta, \Delta^c)$, a set of configurations W is regular if: $(\langle p, \omega \rangle, \theta) \in W$ iff $\omega \in L_p$, where $L_p \subseteq \Gamma^*$ is a regular set for each $p \in P$.*

A regular valuation is a function $\nu : AP \rightarrow 2^{Conf}$ that assigns to each atomic proposition a regular set of configurations. To incorporate regular valuations into the model checking process, we can extend the construction algorithm given in Section 3.2. Let $\mathcal{M}_p$ be a finite automaton that accepts L_p . The extension is straightforward and follows the lines of [23], which consists in popping the whole stack according to the transitions in $\mathcal{M}_p$ when $\mathcal{BP}$ is at the control location of the form $[p, a]$, where $a \in AP$. For a configuration $(\langle p, \gamma_1 \gamma_2 \gamma_3 \# \rangle, \theta)$ in $\mathcal{P}$ and a path in $\mathcal{M}_p$ of the form $q_1 \xrightarrow{\gamma_1} q_2 \xrightarrow{\gamma_2} q_3 \xrightarrow{\gamma_3} q_4$, would correspond to the execution of $\mathcal{BP}$ of the form $(\langle [p, a], \gamma_1 \gamma_2 \gamma_3 \# \rangle, \theta) \Rightarrow (\langle [q_1, a], \gamma_1 \gamma_2 \gamma_3 \# \rangle, \theta) \Rightarrow (\langle [q_2, a], \gamma_2 \gamma_3 \# \rangle, \theta) \Rightarrow (\langle [q_3, a], \gamma_3 \# \rangle, \theta) \Rightarrow (\langle [q_4, a], \# \rangle, \theta)$. Thus, we can show that:

Theorem 5. *We can effectively model check SM-PDS against a BranchCARET formula with regular valuations.*

3.6 Performance concerns

Algorithm 1 presented in Section 3.4 is generic and works with any kind of SM-ABPDS with checkpoints. However, the kind of $\mathcal{BP}$ we construct has some restrictions that allow us to optimize $loop_2$. This optimization is possible because *restore* and *discard* rules are constructed in a specific manner. Let $Q \in \{E, A\}$. Consider Line 8 of the algorithm. It finds paths of the form $(p_j, \theta_j)^k \xrightarrow{em(\theta, w_j)}^* R_j$. If $w_j = \gamma_j \chi_j$, then the algorithm will look first for transitions of the form $(p_j, \theta_j)^k \xrightarrow{\gamma_j} R'_j$, and then, for each $q_j^l \in R'_j$, the algorithm will look for transitions of the form $q_j^l \xrightarrow{\chi_j^\theta} R_j^l$, such that $R_j = \bigcup_l R_j^l$. The transitions of the form $q_j^l \xrightarrow{\chi_j^\theta} R_j^l$ can exist in $\mathcal{A}$ only if they were added on either Line 12 or 15 (we will later explain why the two sets of transitions added by these two cases are disjoint):

- If it was added on Line 15, then $q_j^l = (p, \theta')$ for some $p \in P'$ and $\theta' \subseteq \delta$, and there is $p\chi \hookrightarrow p'\gamma \in \text{discard}$, and $(p', \theta') \xrightarrow{\gamma}^* R_j^l$. By the construction of *discard* rules in $\mathcal{BP}$, which all have the form $[g, \phi][\gamma, p_0, \gamma_0] \hookrightarrow [g, \phi]\gamma, p$ must be equal to p' , and $p = p' = [g, \phi]$ for $g \in P$, $\phi \in Cl(\psi)$, and χ has the form of $[\gamma, g_0, \gamma_0]$ (with the same γ as in right-hand side). Therefore, given that $q_j^l = ([g, \phi], \theta')$ and $\chi = [\gamma, g_0, \gamma_0]$, $q_j^l \xrightarrow{\chi^\theta} R_j^l$ exists iff $q_j^l \xrightarrow{\gamma} R_j^l$ exists.
- On the other hand, if the transition $q_j^l \xrightarrow{\chi^\theta} R_j^l$ was added on Line 12, then $q_j^l = (p, \theta')$, there is $p\chi \hookrightarrow p'\gamma \in \text{restore}$, and $(p', \theta) \xrightarrow{\gamma}^* R_j^l$. By the construction of *restore* rules in $\mathcal{BP}$, p must have a form of $[p^c, \phi]$, p' must be of the form $[p_0, \phi']$ for $p_0 \in P$, and either $\phi' = \phi$, or $\phi = QX^c\phi'$ ($Q \in \{E, A\}$), and χ must be of the form $[\gamma', p_0, \gamma]$. Therefore, given that $q_j^l = ([p^c, QX^c\phi'], \theta')$ and $\chi = [\gamma', p_0, \gamma]$, then $q_j^l \xrightarrow{\chi^\theta} R_j^l$ exists iff $([p_0, \phi'], \theta) \xrightarrow{\gamma} R_j^l$ exists. On the other hand, given that $q_j^l = ([p^c, \phi], \theta')$, $\phi \neq QX^c\phi'$, and $\chi = [\gamma', p_0, \gamma]$, then $q_j^l \xrightarrow{\chi^\theta} R_j^l$ exists iff $([p_0, \phi], \theta) \xrightarrow{\gamma} R_j^l$ exists.

Sets of transitions added by these cases are disjoint since the first one requires that $q_j^l = ([g, \phi], \theta')$ for $g \in P$, and the second one requires that $q_j^l = ([p^c, \phi], \theta')$. Therefore, we can optimize Algorithm 1 as follows:

1. Remove loops on Lines 14 and 11.
2. On line 8, if $em(\theta, w_j) = \gamma_j \chi_j^\theta$ and $\chi_j = [\gamma_j', g_j^0, \gamma_j^0]$, then let $(p_j, \theta_j)^k \xrightarrow{\gamma_j}^* R'_j$. Then, $W_j = \{S \mid ([g, \phi], \theta)^k \in R'_j, ([g, \phi], \theta)^k \xrightarrow{\gamma_j'}^* S\} \cup \{S \mid ([p^c, QX^c\phi], \theta')^k \in R'_j, ([g_j^0, \phi], \theta)^k \xrightarrow{\gamma_j^0}^* S\} \cup \{S \mid ([p^c, \phi], \theta')^k \in R'_j, \phi \neq QX^c\phi', ([g_j^0, \phi], \theta)^k \xrightarrow{\gamma_j^0}^* S\}$.

Therefore, by removing loops on lines 14 and 11, we significantly reduce the amount of transitions added to T . We show the optimized variant of $Y_{\mathcal{BP}}$ computation in Algorithm 2.

Proposition 1. *Algorithm 2 is equivalent to Algorithm 1 under the given construction of $\mathcal{BP}$.*

Algorithm 2 Optimized computation of $Y_{\mathcal{BP}_i}$

Input: An SM-ABPDS $\mathcal{BP}_i = (P', \Gamma', X, \Delta', \text{restore}, \text{discard}, F, \delta)$.

Output: An AMA $\mathcal{A}_i = (Q, G, T, I, \{q_f\})$.

```

1:  $k := 0, T := \{q_f \xrightarrow{\gamma} \{q_f\} \mid \forall \gamma \in \Gamma', \forall p \in P'_i \text{ and } \forall \theta \subseteq \delta, (p, \theta)^0 := q_f\}$ ;
2: repeat(we call this loop1)
3:    $k := k + 1$ ;
4:   For all  $p \in F, \theta \subseteq \delta$ , add  $(p, \theta)^k \xrightarrow{\varepsilon} \{(p, \theta)^{k-1}\} \in T$ ;
5:   repeat(we call this loop2)
6:     for all  $\theta \subseteq \delta, p\gamma \xrightarrow{[\sigma_1, \dots, \sigma_h]} \{p_1 w_1, \dots, p_h w_h\} \in \tau$ , and  $1 \leq j \leq h$  do
7:       let  $\sigma_j = (\rho_j, \rho'_j)$  and  $\theta_j = (\theta \setminus \rho_j) \cup \rho'_j$ ;
8:       if  $w_j = \gamma_j \chi_j, \chi_j = [\gamma'_j, g_j^0, \gamma_j^0]$ , and  $(p_j, \theta_j)^k \xrightarrow{\gamma_j^*} R'_j$  then
9:         let  $W_j = \{S \mid ([g, \phi], \theta)^k \in R'_j, ([g, \phi], \theta)^k \xrightarrow{\gamma_j^*} S\} \cup$ 
           $\{S \mid ([p^c, QX^c\phi], \theta')^k \in R'_j, ([g_j^0, \phi], \theta)^k \xrightarrow{\gamma_j^0} S\} \cup \{S \mid ([p^c, \phi], \theta')^k \in R'_j, \phi \neq$ 
           $QX^c\phi', ([g_j^0, \phi], \theta)^k \xrightarrow{\gamma_j^0} S\}$ ;
10:        else
11:          let  $W_j = \{R_j \mid \rho_j \subseteq \theta, (p_j, \theta_j)^k \xrightarrow{w_j} R_j\}$ ;
12:        end if
13:         $T := T \cup \{(p, \theta)^k \xrightarrow{\gamma} \bigcup_{R \in W} R \mid W \in \prod_{j=1}^h W_j\}$ ;
14:      end for
15:    until No new transitions can be added;
16:    For all  $p \in F, \theta \subseteq \delta$ , remove  $(p, \theta)^k \xrightarrow{\varepsilon} \{(p, \theta)^{k-1}\}$  from  $T$ ;
17:    For all  $p \in P', \theta \subseteq \delta, \gamma \in G, R \subseteq Q$ , replace  $(p, \theta)^k \xrightarrow{\gamma} R$  by  $(p, \theta)^k \xrightarrow{\gamma} pr^k(R)$ ;
18: until  $k > 1$  and  $\forall p \in P', \gamma \in G, \theta \subseteq \delta, R \subseteq P' \times 2^\delta \times \{k\} \cup \{q_f\}, (p, \theta)^k \xrightarrow{\gamma} R \in$ 
       $T \iff (p, \theta)^{k-1} \xrightarrow{\gamma} pr^{-1}(R) \in T$ ;

```

4 Experiments

We have implemented our algorithm using Zig programming language and performed practical experiments.

4.1 Our Algorithm vs. Naive Approach

It was shown that given an SM-PDS, one can construct an equivalent standard non self-modifying PDS [24]. And an efficient algorithm for BranchCARET model checking of standard PDS was proposed in [16, 21]. Therefore, a naive approach for BranchCARET model checking of SM-PDS would be to construct an equivalent PDS, and use the existing algorithms. However, this approach is not

efficient. We have implemented the naive approach and compared it with our algorithm. We generated a random SM-PDS $\mathcal{P}$ and a random BranchCARET formula ψ . Then, we ran both algorithms to model check $\mathcal{P}$ against ψ on a machine with Intel Core i3-4150 CPU @ 3.50GHz with 10 GB available memory. We recorded the execution time and the maximum memory consumed during execution. We repeated this process multiple times and presented the results in Table 1. Our algorithm shows significant improvement in both speed and memory consumption (up to 30x faster and 40x less memory required). We highlighted in the table the cases showing significant improvements in memory or time. Moreover, when we consider large SM-PDS models (≈ 800 rules) or long BranchCARET formulas (≈ 7 -9 operators), the naive approach fails either because of timeout, or because of lack of memory, while our algorithm finishes in seconds.

4.2 Application for Malware Detection

We have applied our algorithm for malware detection. For this, we gathered malware samples from public databases such as Malware Bazaar [1] and Virus Share [13]. We focus on binaries for x86 architecture under Windows, since this is the most common target for malware. Then, we have built a tool based on our algorithm that performs malware detection. It translates binary files into Control Flow Graphs (CFGs) using ANGR [22]. The obtained CFGs are translated into SM-PDS models using technique presented by [23, 24]. The technique models executable binary addresses as control locations and binary instructions as SM-PDS rules. For example, `call` instructions are modelled as *call* rules, `ret` instructions are modelled as *ret* rules, self-modifying instructions are modelled as self-modifying rules, and other instructions as *int* rules.

Then, we specified several malicious behaviours using BranchCARET formulas. We executed the tool to check if the SM-PDS model of the binary satisfies the given BranchCARET formula. If it does, i.e. the model contains the malicious behaviour, the tool marked the binary as malicious. We illustrated the detected malware in Table 2. First, let us present BranchCARET formulas for malicious behaviour we used. We use operators of the forms $\mathbf{QG}^b$ and $\mathbf{QF}^b$, where $Q \in \{E, A\}$ and $b \in \{g, a, c\}$, to represent *globally* ($\mathbf{QG}^b\phi = \mathbf{Q}[False\mathbf{R}^b\phi]$) and *eventually* ($\mathbf{QF}^b\phi = \mathbf{Q}[True\mathbf{U}^b\phi]$) operators. Furthermore, it is important to recognize system calls in the BranchCARET formula. Sometimes, malware obfuscates calls to system functions using a chain of `jump` instructions, e.g. as in this code:

```
d0:  push 0
d1:  call [d]
d2:  ...
d :  jmp [GetModuleFileNameA]
```

The code obfuscates instruction `call [GetModuleFileNameA]` that calls Windows function *GetModuleFileNameA*. Instead, it calls a non-existent function with address `d`, while the address `d` contains a single jump instruction to the function *GetModuleFileNameA*.

Table 1. Comparison of performance of our approach vs. PDS approach ($|\Delta| + |\Delta^c|$ are numbers of standard and self-modifying rules respectively, $|\psi|$ is the size of Branch-CARET formula, OOM - out of memory).

		Our approach		PDS approach	
$ \Delta + \Delta^c $	$ \psi $	Time, s	Memory	T, s	Memory
74 + 7	9	1.754s	37.312 MB	21.883s	591.418 MB
117 + 7	3	0.262s	10.426 MB	6.036s	362.770 MB
132 + 8	5	0.977s	23.273 MB	17.755s	540.113 MB
113 + 11	3	0.6s	14.738 MB	19.386s	721.586 MB
75 + 12	3	0.25s	16.027 MB	6.096s	400.164 MB
104 + 12	9	4.41s	73.320 MB	46.727s	1.517 GB
114 + 12	5	0.438s	18.402 MB	17.145s	837.953 MB
81 + 14	5	0.636s	23.898 MB	12.369s	687.258 MB
75 + 15	7	2.1s	40.711 MB	22.208s	830.094 MB
93 + 15	5	0.385s	17.191 MB	7.123s	651.023 MB
90 + 17	9	2.374s	53.766 MB	17.505s	766.270 MB
78 + 19	5	0.365s	14.930 MB	9.664s	601.031 MB
78 + 20	5	2.25s	70.480 MB	16.578s	920.098 MB
99 + 20	5	1.065s	37.344 MB	24.409s	1.613 GB
104 + 11	9	0.781s	18.879 MB	Timeout	Timeout
108 + 13	7	9.301s	91.605 MB	OOM	OOM
105 + 18	7	0.727s	27.301 MB	OOM	OOM
68 + 20	9	9.757s	35.293 MB	Timeout	Timeout
89 + 19	7	1.859s	42.664 MB	Timeout	Timeout
118 + 20	9	11.547s	116.238 MB	OOM	OOM
86 + 19	9	2.732s	52.227 MB	Timeout	Timeout
1083 + 18	5	88.416s	4.246 GB	OOM	OOM
1224 + 16	9	112.57s	3.080 GB	OOM	OOM
677 + 19	3	13.685s	661.395 MB	OOM	OOM
714 + 12	3	11.508s	748.539 MB	43.048s	3.199 GB
1390 + 8	7	43.385s	1.589 GB	OOM	OOM
851 + 14	9	135.278s	4.179 GB	OOM	OOM
1280 + 17	5	64.454s	3.873 GB	OOM	OOM

Table 2. Malware detection results.

Malware	# of rules	ψ	Time
Virus.Win32.Spit.c.exe	1332	ψ_s	581.12s
Trojan-Downloader.Win32.Small.axw.exe	1376	ψ_s	89.28s
P2P-Worm.Win32.Compux.a.exe	355	ψ_s	19.46s
IRC-Worm.Win32.Azaco.c.exe	425	ψ_s	23.06s
Trojan-Downloader.Win32.Small.bdw.exe	1568	ψ_s	124.15s
Virus.Win32.Mooder.l.exe	623	ψ_s	59.90s
Virus.Win32.Smog.e.exe	127	ψ_s	2.76s
Backdoor.Win32.SubSeven.22.b2.exe	2238	ψ_s	590.01s
Trojan-Downloader.Win32.Small.hj.exe	1729	ψ_t	387.94s
Trojan-Downloader.Win32.Small.xg.exe	743	ψ_t	806.43s
Trojan-Downloader.Win32.Small.cho.exe	659	ψ_t	241.60s
Trojan.Win32.KillAV.bx.exe	457	ψ_k	18.85s
Trojan-Downloader.Win32.Small.cho.exe	659	ψ_t	217.29s
Virus.Win32.Mooder.i.exe	623	ψ_s	32.55s
Trojan-Spy.Win32.Cusrev.a.exe	557	ψ_k	15.31s
Virus.Win32.Yopper.exe	92	ψ_s	1.15s
Virus.Win32.Invictus.exe	140	ψ_s	1.05s
Virus.Win32.Morgoth.2560.exe	106	ψ_s	0.85s
Virus.Win32.Mooder.a.exe	623	ψ_s	32.94s
Virus.Win32.Kanban.a.exe	140	ψ_s	1.06s
Trojan-Downloader.Win32.Botten.exe	726	ψ_t	2785.14s
Trojan.Win32.StartPage.ay.exe	3489	ψ_k	252.44s
Trojan-Downloader.Win32.Sherlol.exe	1294	ψ_t	1360.80s
Virus.Win32.Mooder.g.exe	236	ψ_s	4.08s
Trojan.Win32.LOADER.WPW.exe	298	ψ_t	13.80s
Trojan-Downloader.Win32.Small.bdc.exe	539	ψ_t	39.18s
Trojan-Downloader.Win32.Small.ef.exe	1615	ψ_t	269.74s
Trojan-Downloader.Win32.Harnig.e.exe	1921	ψ_t	866.80s
Trojan.Win32.KillAV.bb.exe	657	ψ_k	163.42s
Trojan-Dropper.Win32.Small.ir.exe	650	ψ_k	33.48s
Virus.Win32.Mooder.d.exe	236	ψ_s	6.70s
Virus.Win32.Mooder.c.exe	623	ψ_s	53.07s
Backdoor.Win32.DTR.a.exe	455	ψ_t	135.22s
Trojan-Downloader.Win32.Harnig.a.exe	2396	ψ_t	1589.98s
Trojan.Win32.Avkillah.a.exe	4986	ψ_k	441.23s
Virus.Win32.Mooder.b.exe	623	ψ_s	52.52s
Trojan-Dropper.Win32.Small.kj.exe	1090	ψ_k	3471.64s
Trojan-Downloader.Win32.Small.fn.exe	1760	ψ_t	386.02s
Trojan.Win32.KillAV.bc.exe	457	ψ_k	80.71s
Virus.Win32.Fela.8192.exe	97	ψ_s	0.72s
Trojan-Downloader.Win32.Aphex.10.d.exe	662	ψ_k	167.18s
Trojan-Downloader.Win32.Harnig.av.exe	1197	ψ_t	940.48s
Trojan.Win32.OpenPort.c.exe	373	ψ_t	101.40s

Thus, for some function $func$, let $call(func) = call \wedge \mathbf{AX}^g \mathbf{A}[jmp \mathbf{U}^g func]$ be a shortcut formula specifying that the Windows function $func$ is called. It means that the current instruction is `call`, and $\mathbf{AX}^g$ refers to the called address. Then, $\mathbf{A}[jmp \mathbf{U}^g func]$ means that all paths lead to the address of $func$, and all instruction on the paths are `jmp` instructions.

Trojan downloader behaviour Trojan downloaders are programs that are designed to deliver other malicious files to the target computer. They download the malicious files from the network and execute them immediately. To download files, programs must open a socket using `socket` function, and then receive the files using `recv` function, which takes the opened socket as a parameter. Then, the downloaded file gets executed using the `CreateProcess` function. This behaviour can be described using the following formula (we consider the disjunction $\bigvee_{d \in \Gamma}$ over all values d possibly located on the stack):

$$\psi_t = \bigvee_{d \in \Gamma} \mathbf{EG}^g \left[\mathbf{EF}^g [call(socket) \wedge \mathbf{EX}^a eax = d \wedge \mathbf{EF}^a [call(recv) \wedge d\Gamma^* \wedge \mathbf{EF}^g call(CreateProcess)]] \right]$$

Intuitively, this formula describes that there is a loop, such that in the future, $call(socket) \wedge \mathbf{EX}^a eax = d \wedge \mathbf{EF}^a [call(recv) \wedge d\Gamma^* \wedge \mathbf{EF}^g call(CreateProcess)]$ is satisfied. Subformula $call(socket) \wedge \mathbf{EX}^a eax = d$ means that the `socket` function is called, and at the return location of `socket` (i.e. at the abstract successor of the call), the value of the register `eax` is equal to d . Usually, the value of the `eax` register right after returning from a function contains the return value of the function. That is, the subformula means that the return value of the `socket` function (the opened socket) is equal to d . Then, the subformula $\mathbf{EF}^a [call(recv) \wedge d\Gamma^* \wedge \mathbf{EF}^g call(CreateProcess)]$ means that in the future of the same procedural context (in which `socket` was called), the subformula $call(recv) \wedge d\Gamma^* \wedge \mathbf{EF}^g call(CreateProcess)$ is satisfied. The subformula $call(recv)$ states that the `recv` function is called, and the subformula $d\Gamma^*$ means that the stack contains d on the top (we can reason about stack contents in such a way thanks to regular valuations introduced in Section 3.5). Since parameters of functions are passed on the stack, these two formulas together mean that the `recv` function is called with d as parameter. Then, subformula $\mathbf{EF}^g call(CreateProcess)$ states that the `CreateProcess` function will be called in the future.

Spyware behaviour [21] introduced the BCaRet formula describing spyware behaviour. Spyware is a type of malware that collects files matching certain conditions. To perform file searching, spyware first calls function `FindFirstFileA`, which searches files in the given folder. If the call to `FindFirstFileA` fails, then spyware obtains the reason for the failure using function `GetLastError`. Otherwise, the `FindFirstFileA` function returns a search handle d that points to the

first found item. The found item may be a folder or a file. If d is a folder, the spyware calls *FindFirstFileA* again on the found folder to search for files inside of the subfolder. If d is a file, the spyware obtains a handle to the next file matching the conditions using the *FindNextFileA* function with d as parameter. This behaviour can be described as follows:

$$\psi_s = \bigvee_{d \in \Gamma} \mathbf{EF}^g [call(FindFirstFileA) \wedge \mathbf{EX}^a eax = d \wedge \mathbf{AF}^a [call(GetLastError) \vee call(FindFirstFileA) \vee (call(FindNextFileA) \wedge d\Gamma^*)]]$$

This formula states that in the future, formula $call(FindFirstFileA) \wedge \mathbf{EX}^a eax = d \wedge \mathbf{AF}^a [call(GetLastError) \vee call(FindFirstFileA) \vee (call(FindNextFileA) \wedge d\Gamma^*)]$ is satisfied. The subformula $call(FindFirstFileA) \wedge \mathbf{EX}^a eax = d$ states that the *FindFirstFileA* function is called, and at the return location from *FindFirstFileA*, the value of the *eax* register is equal to d . Then, subformula $\mathbf{AF}^a [call(GetLastError) \vee call(FindFirstFileA) \vee (call(FindNextFileA) \wedge d\Gamma^*)]$ states that in all possible futures, in the same procedure context, either the *GetLastError* function is called, the *FindFirstFileA* function is called, or formula $call(FindNextFileA) \wedge d\Gamma^*$ is satisfied. The formula $call(FindNextFileA) \wedge d\Gamma^*$ states that the *FindNextFileA* function is called with d as its parameter.

Anti-antivirus behaviour Some malware attempts to evade antivirus programs by first shutting down the antivirus and then executing the malicious payload. To terminate the antivirus process, first, malware obtains the list of currently running processes using the *CreateToolhelp32Snapshot* function, which returns a handle d to the list. Then, it calls the *Process32First* function with d as parameter to get the first process from the list. Then, it iterates over the list by calling the *Process32Next* function with d as parameter in a loop. Then, if it finds the antivirus program, it calls the *OpenProcess* function to obtain access to the antivirus program. Then, the malware shuts down the antivirus by calling the *TerminateProcess* function. Such behaviour can be described using the following formula:

$$\begin{aligned} \psi_k = \bigvee_{d \in \Gamma} \mathbf{EF}^g & \left[call(CreateToolhelp32Snapshot) \wedge \mathbf{EX}^a eax = d \wedge \right. \\ & \mathbf{EF}^g [call(Process32First) \wedge d\Gamma^* \wedge \mathbf{AG}^g \mathbf{EF}^g [call(Process32Next) \wedge d\Gamma^* \wedge \\ & \left. \mathbf{EF}^g [call(OpenProcess) \wedge \mathbf{AF}^a call(TerminateProcess)]]]] \right] \end{aligned}$$

The formula states that eventually, the subformula $call(CreateToolhelp32Snapshot) \wedge \mathbf{EX}^a eax = d$ will be satisfied. This subformula states that the *CreateToolhelp32Snapshot* function is called, and the return value of the function is equal to d . Then, the subformula $call(Process32First) \wedge d\Gamma^*$ states that the *Process32First* function is called with d as parameter. Then, the subformula $\mathbf{AG}^g \mathbf{EF}^g [call(Process32Next) \wedge d\Gamma^* \wedge \mathbf{EF}^g [call(OpenProcess) \wedge \mathbf{AF}^a call(TerminateProcess)]]$ states that there

is a loop ($\mathbf{AG}^g\mathbf{EF}^g$), in which the function *Process32Next* is called with *d* as parameter, and in the future, the subformula $\mathbf{EF}^g[\text{call}(\text{OpenProcess}) \wedge \mathbf{AF}^a \text{call}(\text{TerminateProcess})]$ is satisfied. This subformula describes that in the future, the *OpenProcess* function is called, and after that, in all possible runs the *TerminateProcess* function is called within the same procedural context.

5 Conclusion

In this work, we considered BranchCARET model checking of SM-PDS, which is a natural model of self-modifying binary code. We proposed a *direct* and *efficient* algorithm for model checking. We performed practical experiments, showing that our approach is faster and requires less memory than the naive approach, which consists in translating the SM-PDS into a standard PDS. Furthermore, we were able to detect several malwares by performing model checking of SM-PDS models of binary programs against BranchCARET formulas specifying malicious behaviours. Although our approach focuses on SM-PDS, a similar approach can be applied for BranchCARET model checking of symbolic PDS because the problem presents the same challenges as SM-PDS, i.e. an implicit state that is modified along the execution, but not explicitly stated in the pushdown rules.

References

1. abuse.ch: Malware Bazaar. <https://bazaar.abuse.ch/about/>, accessed: 2025-04-20
2. Alur, R., Etessami, K., Madhusudan, P.: A temporal logic of nested calls and returns. In: International Conference on Tools and Algorithms for the Construction and Analysis of Systems. pp. 467–481. Springer (2004)
3. Anckaert, B., Madou, M., De Bosschere, K.: A model for self-modifying code. In: Information Hiding: 8th International Workshop, IH 2006, Alexandria, VA, USA, July 10–12, 2006. Revised Selected Papers 8. pp. 232–248. Springer (2007)
4. Battista, P., Mercaldo, F., Nardone, V., Santone, A., Visaggio, C.A., et al.: Identification of android malware families with model checking. In: ICISSP. pp. 542–547 (2016)
5. Beaucamps, P., Gnaedig, I., Marion, J.Y.: Abstraction-based malware analysis using rewriting and model checking. In: Computer Security—ESORICS 2012: 17th European Symposium on Research in Computer Security, Pisa, Italy, September 10–12, 2012. Proceedings 17. pp. 806–823. Springer (2012)
6. Blackham, B., Heiser, G.: Sequoll: a framework for model checking binaries. In: 2013 IEEE 19th Real-Time and Embedded Technology and Applications Symposium (RTAS). pp. 97–106. IEEE (2013)
7. Blazy, S., Laporte, V., Pichardie, D.: Verified abstract interpretation techniques for disassembling low-level self-modifying code. *Journal of Automated Reasoning* **56**, 283–308 (2016)
8. Bonfante, G., Fernandez, J., Marion, J.Y., Rouxel, B., Sabatier, F., Thierry, A.: Codisasm: Medium scale concatic disassembly of self-modifying binaries with overlapping instructions. In: Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security. pp. 745–756 (2015)

9. Bonfante, G., Marion, J.Y., Reynaud-Plantey, D.: A computability perspective on self-modifying programs. In: 2009 Seventh IEEE International Conference on Software Engineering and Formal Methods. pp. 231–239. IEEE (2009)
10. Cai, H., Shao, Z., Vaynberg, A.: Certified self-modifying code. In: Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation. pp. 66–77 (2007)
11. Chipounov, V., Kuznetsov, V., Candea, G.: S2e: A platform for in-vivo multi-path analysis of software systems. *Acm Sigplan Notices* **46**(3), 265–278 (2011)
12. Coogan, S.K., Debray, K.P., Townsend, G.M.: On the semantics of self-unpacking malware code. Citeseer, Tech. Rep. (2008)
13. Corvus Forensics: Virus Share. <https://virusshare.com/about>, accessed: 2025-04-20
14. David, R., Bardin, S., Ta, T.D., Mounier, L., Feist, J., Potet, M.L., Marion, J.Y.: Binsec/se: A dynamic symbolic execution toolkit for binary-level analysis. In: 2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER). vol. 1, pp. 653–656. IEEE (2016)
15. Gorna, K., Iooss, N., Seurin, Y., Khatoun, R.: Zorya: Automated concolic execution of single-threaded go binaries. Symposium on Applied Computing (2026)
16. Gutsfeld, J.O., Müller-Olm, M., Nordhoff, B.: A branching time variant of caret. In: International Symposium on Model Checking Software. pp. 153–170. Springer (2018)
17. Kinder, J., Katzenbeisser, S., Schallhart, C., Veith, H.: Detecting malicious code by model checking. In: Detection of Intrusions and Malware, and Vulnerability Assessment: Second International Conference, DIMVA 2005, Vienna, Austria, July 7-8, 2005. Proceedings 2. pp. 174–187. Springer (2005)
18. King, J.C.: Symbolic execution and program testing. *Communications of the ACM* **19**(7), 385–394 (1976)
19. Korczynski, D.: Repeconstruct: reconstructing binaries with self-modifying code and import address table destruction. In: 2016 11th International Conference on Malicious and Unwanted Software (MALWARE). pp. 1–8. IEEE (2016)
20. Nguyen, H.V., Touili, T.: CARET model checking for malware detection. In: Proceedings of the 24th ACM SIGSOFT international SPIN symposium on model checking of software. pp. 152–161 (2017)
21. Nguyen, H.V., Touili, T.: Branching temporal logic of calls and returns for pushdown systems. In: International Conference on Integrated Formal Methods. pp. 326–345. Springer (2018)
22. Shoshitaishvili, Y., Wang, R., Salls, C., Stephens, N., Polino, M., Dutcher, A., Grosen, J., Feng, S., Hauser, C., Kruegel, C., et al.: SoK:(state of) the art of war: Offensive techniques in binary analysis. In: 2016 IEEE symposium on security and privacy (SP). pp. 138–157. IEEE (2016)
23. Song, F., Touili, T.: Efficient CTL model-checking for pushdown systems. *Theoretical Computer Science* **549**, 127–145 (2014)
24. Touili, T., Ye, X.: LTL model checking of self modifying code. *Formal Methods in System Design* **60**(2), 195–227 (2022)
25. Touili, T., Zhangeldinov, O.: CTL model checking of concurrent self modifying code. In: Symposium on Applied Computing (2026)

A Proof of Theorem 1

Proof. Let us define a function $Assoc$, which associates a formula $\phi \in Cl(\psi)$ and the i -th step on a run π of $\mathcal{P}$ into a configuration $(\langle [p, \phi], w' \rangle, \theta)$ of $\mathcal{BP}$. $Assoc(\phi, \pi, i) = (\langle [p, \phi], w' \rangle, \theta)$, s.t. if $w = \gamma_1 \dots \gamma_n$, then $w' = \gamma'_1 \dots \gamma'_n$, where for $1 \leq i \leq n$, $\gamma'_i = \lfloor \gamma_i, p_0, \gamma_0 \rfloor$ if γ_i was pushed by a rule of form $p_0 \gamma_0 \xrightarrow{call} p' \gamma'_0 \gamma_i$, and $\gamma'_i = \gamma_i$ otherwise.

We prove the theorem by showing that for any run π in $\mathcal{P}$, $(\pi, i) \models \phi$ iff there is an accepting run in $\mathcal{BP}$ starting from $Assoc(\phi, \pi, i)$. The proof is by induction on ϕ :

– **Base:**

1. $\phi = a \in AP$. Let $\pi(i) = (\langle p, \gamma w \rangle, \theta)$. $(\pi, i) \models a$ iff $p \in \nu(a)$ iff $\exists [p, a] \gamma \xrightarrow{[\top]} \{[p, a] \gamma\} \in \Delta'$ (by item 1) iff there is an run visiting $[p, a]$ infinitely many times iff there is an accepting run starting from $Assoc(a, \pi, i)$ (the run is accepting because $[p, a] \in F$)
2. $\phi = \neg a \in AP$. Let $\pi(i) = (\langle p, \gamma w \rangle, \theta)$. $(\pi, i) \models \neg a$ iff $p \notin \nu(a)$ iff $\exists [p, \neg a] \gamma \xrightarrow{[\top]} \{[p, \neg a] \gamma\} \in \Delta'$ (by item 2) iff there is an run visiting $[p, \neg a]$ infinitely many times iff there is an accepting run starting from $Assoc(\neg a, \pi, i)$ (the run is accepting because $[p, \neg a] \in F$)

– **Step:**

1. $\phi = \phi_1 \vee \phi_2$. $(\pi, i) \models \phi$ iff $(\pi, i) \models \phi_1$ or $(\pi, i) \models \phi_2$ iff there is an accepting run from $Assoc(\phi_1, \pi, i)$ or $Assoc(\phi_2, \pi, i)$ (by induction hypothesis) iff there is an accepting run from $Assoc(\phi, \pi, i)$ (from item 3).
2. $\phi = \phi_1 \wedge \phi_2$. Similar to case 1 using item 4.
3. $\phi = EX^g \phi'$. Suppose $\pi(i) = (\langle p, \gamma w \rangle, \theta)$. $(\pi, i) \models \phi$ iff $\exists \pi' \in Ext(\pi), \pi'(i+1) = (\langle p', w \omega \rangle, \theta')$ and $(\pi', i+1) \models \phi'$, iff (1) $\exists p \gamma \xrightarrow{(\rho, \rho')} p' w \in \tau$, s.t. $\theta' = (\theta \setminus \rho) \cup \rho'$, and (2) there is an accepting run starting from $Assoc(\phi', \pi', i+1)$ (by induction hypothesis). (1) iff $\exists [p, \phi] \gamma \xrightarrow{[(\rho, \rho')]} p' w' \in \Delta'$ iff $Assoc(\phi, \pi, i) \implies \{Assoc(\phi', \pi', i+1)\}$ iff there is an accepting run from $Assoc(\phi, \pi, i)$ (from (2)).
4. $\phi = AX^g \phi'$. The proof is similar to case 3.
5. $\phi = EX^a \phi'$. Suppose $\pi(i) = (\langle p, \gamma w \rangle, \theta)$. $(\pi, i) \models \phi$ iff $\exists \pi' \in Ext(\pi), \pi'(next_{\pi}^a(i)) = (\langle p', w \omega \rangle, \theta')$ and $(\pi', next_{\pi}^a(i)) \models \phi'$, iff either:
 - $next_{\pi}^a(i) = i+1$ iff (1): $\exists p \gamma \xrightarrow{(\rho, \rho')}_{int} p' w \in \tau$, s.t. $\theta' = (\theta \setminus \rho) \cup \rho'$, and (2): there is an accepting run starting from $Assoc(\phi', \pi', i+1)$ (by induction hypothesis). (1) iff $\exists [p, \phi] \gamma \xrightarrow{[(\rho, \rho')]} p' w' \in \Delta'$ iff $Assoc(\phi, \pi, i) \implies \{Assoc(\phi', \pi', i+1)\}$ iff there is an accepting run from $Assoc(\phi, \pi, i)$ (from (2)); or
 - $next_{\pi}^a(i) = k > i+1$ iff (1): $\exists p \gamma \xrightarrow{(\rho, \rho')}_{call} p' \gamma_1 \gamma_2$ and (2): there is a path $(\langle p', \gamma_1 \gamma_2 \omega \rangle, \theta) \implies^* (\langle p'', \gamma_2 \omega \rangle, \theta')$, s.t. $\pi'(next_{\pi}^a(i)) = (\langle p'', \gamma_2 \omega \rangle, \theta')$. (1) iff $\exists [p, \phi] \gamma \xrightarrow{[(\rho, \rho')]} \{[p', \phi]^r \gamma_1 \gamma_2^r\} \in \Delta'$ (from item

- 7) iff $(\langle [p, \phi], \gamma\omega' \rangle, \theta) \implies \{(\langle [p', \phi]^r, \gamma_1\gamma_2^r\omega' \rangle, \theta)\}$. Then, (2) iff (3): $(\langle [p', \phi]^r\gamma_1\gamma_2^r\omega' \rangle, \theta) \implies \{(\langle [p'', \phi]^r\gamma_2^r\omega' \rangle, \theta')\}$. Then, from item 11, (4): $(\langle [p'', \phi]^r\gamma_2^r\omega' \rangle, \theta') \implies \{(\langle [p'', \phi']\gamma_2\omega' \rangle, \theta')\}$. Thus, ((1) and (2)) iff ((3) and (4)) iff $\text{Assoc}(\phi, \pi, i) \implies^* \text{Assoc}(\phi', \pi', \text{next}_{\pi'}^a(i))$ iff there is an accepting run from $\text{Assoc}(\phi, \pi, i)$.
6. $\phi = AX^a\phi'$. The proof is similar to case 5.
7. $\phi = E[\phi_1 U^g \phi_2]$. $(\pi, i) \models \phi$ iff (1): $\exists \pi' \in \text{Ext}(\pi, i)$, s.t. $\exists k \geq i$, $(\pi', k) \models \phi_2$ and for each $i \leq j < k$, $(\pi', j) \models \phi_1$. We prove by induction on $k - i$:
- **Base** $k = i$. (1) iff $(\pi, i) \models \phi_2$ iff there is an accepting run starting from $\text{Assoc}(\phi_2, \pi, i) = (\langle [p, \phi_2], w \rangle, \theta)$ (from induction on ϕ) iff there is an accepting run from $(\langle [p, \phi], w \rangle, \theta)$ (from rule added by item 12).
 - **Step** $k > i$. (1) iff (2): $(\pi, i) \models \phi_1$ and (3): $(\pi', i+1) \models \phi$. (2) iff there is an accepting run from $\text{Assoc}(\phi_1, \pi, i) = (\langle [p, \phi_1], \gamma\omega \rangle, \theta)$ (by induction on ϕ). (3) iff there is an accepting run from $\text{Assoc}(\phi, \pi', i+1) = (\langle [p', \phi], w\omega \rangle, \theta')$ (by induction on $k - i$). $\exists \pi' \in \text{Ext}(\pi, i)$ and $\pi'(i+1) = (\langle p', w'\omega \rangle, \theta)$ iff $\exists p\gamma \xrightarrow{(\rho, \rho')} p w'$ iff $\exists [p, \phi]\gamma \xrightarrow{(\rho, \rho')} \{[p, \phi_1]\gamma, [p', \phi]w\}$ (from rule added by item 14) iff there is an accepting run from $\text{Assoc}(\phi, \pi, i)$ and there is no accepting runs from $\text{Assoc}(\phi_2, \pi, i)$.
8. $\phi = A[\phi_1 U^g \phi_2]$. The proof similar to case 7.
9. $\phi = E[\phi_1 U^a \phi_2]$. $(\pi, i) \models \phi$ iff (1): $\exists \pi' \in \text{Ext}(\pi, i)$, s.t. there is a sequence $s = s_0 s_1 \dots s_k$, $s_0 = i$, for any $j \geq 0$, $s_{j+1} = \text{next}_{\pi'}^a(s_j)$, $(\pi', s_k) \models \phi_2$ and for each $0 \leq j < k$, $(\pi', s_j) \models \phi_1$. We prove by induction on k :
- **Base** $k = 0$. (1) iff $(\pi, i) \models \phi_2$ iff there is an accepting run starting from $\text{Assoc}(\phi_2, \pi, i) = (\langle [p, \phi_2], w \rangle, \theta)$ (from induction on ϕ) iff there is an accepting run from $(\langle [p, \phi], w \rangle, \theta)$ (from rule added by item 12).
 - **Step** $k > 0$. (1) iff (2): $(\pi, i) \models \phi_1$ and (3): $(\pi', \text{next}_{\pi'}^a(i)) \models \phi$. (2) iff there is an accepting run from $\text{Assoc}(\phi_1, \pi, i) = (\langle [p, \phi_1], \gamma\omega' \rangle, \theta)$ (by induction on ϕ). (3) iff there is an accepting run from $\text{Assoc}(\phi, \pi', \text{next}_{\pi'}^a(i)) = (\langle [p', \phi], w'\omega \rangle, \theta')$ (by induction on k). $\exists \pi' \in \text{Ext}(\pi, i)$ and $\pi'(\text{next}_{\pi'}^a(i)) = (\langle p', w'\omega \rangle, \theta)$ iff either:
 - * $\exists p\gamma \xrightarrow{(\rho, \rho')}_{\text{int}} p w'$ iff $\exists [p, \phi]\gamma \xrightarrow{(\rho, \rho')} \{[p, \phi_1]\gamma, [p', \phi]w\}$ (from rule added by item 14) iff there is an accepting run from $\text{Assoc}(\phi, \pi, i)$,
 - or
 - * $\exists p\gamma \xrightarrow{(\rho, \rho')}_{\text{call}} p''\gamma_1\gamma_2$ and (2): there is a path $(\langle p'', \gamma_1\gamma_2\omega \rangle, \theta) \implies^* (\langle p', \gamma_2\omega \rangle, \theta')$, s.t. $w' = \gamma_2$. (1) iff $\exists [p, \phi]\gamma \xrightarrow{[(\rho, \rho')]} \{[p'', \phi]^r\gamma_1\gamma_2^r\} \in \Delta'$ (from item 16) iff $(\langle [p, \phi], \gamma\omega' \rangle, \theta) \implies \{(\langle [p', \phi]^r, \gamma_1\gamma_2^r\omega' \rangle, \theta)\}$. Then, (2) iff (3): $(\langle [p'', \phi]^r\gamma_1\gamma_2^r\omega' \rangle, \theta) \implies \{(\langle [p', \phi]^r\gamma_2^r\omega' \rangle, \theta')\}$. Then, from item 18, (4): $(\langle [p', \phi]^r\gamma_2^r\omega' \rangle, \theta') \implies \{(\langle [p', \phi]\gamma_2\omega' \rangle, \theta')\}$. Thus, ((1) and (2)) iff ((3) and (4)) iff $\text{Assoc}(\phi, \pi, i) \implies^* \text{Assoc}(\phi, \pi', \text{next}_{\pi'}^a(i))$ iff there is an accepting run from $\text{Assoc}(\phi, \pi, i)$.
10. $\phi = A[\phi_1 U^a \phi_2]$. The proof similar to case 9.

11. $\phi = E[\phi_1 R^g \phi_2]$. $(\pi, i) \models \phi$ iff (1): $\exists \pi' \in Ext(\pi, i)$, s.t. $\forall k \geq i$, $(\pi', k) \not\models \phi_2$ implies that $\exists j < k$, $(\pi', j) \models \phi_1$. There are two cases: either $\forall k \geq i$, $(\pi', k) \models \phi_2$, or there is $k \geq i$, s.t. $(\pi, k) \models \phi_1 \wedge \phi_2$. The latter case is proven similarly to case 7.

What is left to show is that $\forall k \geq i$, $(\pi', k) \models \phi_2$ iff $\forall k \geq i$, there is an accepting run from $Assoc(\phi, \pi', k)$. We will prove in both directions:

- ($\implies$) We construct the accepting run from $Assoc(\phi, \pi, i) = (\langle [p, \phi], \gamma\omega' \rangle, \theta)$. We can construct a tree with the root $Assoc(\phi, \pi, i)$, and such that every branch is accepting using rules added by item 19. There is an infinite branch b with nodes of the form $(\langle [p', \phi], \omega' \rangle, \theta')$. From each of these nodes, there is an infinite branch $b_{\phi_2} = (\langle [p', \phi_2], \omega' \rangle, \theta')$. From induction on ϕ , b_{ϕ_2} is accepting. Moreover, b is also accepting because control locations of the form $[p', \phi]$ are accepting.
- ($\impliedby$) The accepting run can be constructed using rules from item 19 and represented by the tree with the root $Assoc(\phi, \pi, i)$, and each node of the form $Assoc(\phi, \pi', k)$ for $k \geq i$ having children $Assoc(\phi_2, \pi', k)$ and $Assoc(\phi, \pi', k+1)$. By induction hypothesis, $(\pi', k) \models \phi_2 \forall k \geq i$.

12. $\phi = A[\phi_1 R^g \phi_2]$. The proof similar to case 11.
13. $\phi = E[\phi_1 R^a \phi_2]$. $(\pi, i) \models \phi$ iff (1): $\exists \pi' \in Ext(\pi, i)$, s.t. for a sequence $s = s_0 s_1 \dots$, where $s_0 = i$, we have that $\forall k \geq 0$, $(\pi', s_k) \not\models \phi_2$ implies that $\exists j < k$, $(\pi', s_j) \models \phi_1$. There are two cases: either $\forall k \geq 0$, $(\pi', s_k) \models \phi_2$, or there is $k \geq 0$, s.t. $(\pi, s_k) \models \phi_1 \wedge \phi_2$. The latter case is proven similarly to case 9.

What is left to show is that $\forall k \geq 0$, $(\pi', s_k) \models \phi_2$ iff $\forall k \geq 0$, there is an accepting run from $Assoc(\phi, \pi', s_k)$. There are two cases: either s is finite, or s is infinite. We will prove in both directions:

- ($\implies$) We construct the accepting run from $Assoc(\phi, \pi, i) = (\langle [p, \phi], \gamma\omega' \rangle, \theta)$. We can construct a tree with the root $Assoc(\phi, \pi, i)$, and such that every branch is accepting using rules added by item 21 and item 8. If s is infinite, then there is an infinite branch b with infinitely many nodes of the form $(\langle [p', \phi], \omega' \rangle, \theta')$. From each of these nodes, there is an infinite branch $b_{\phi_2} = (\langle [p', \phi_2], \omega' \rangle, \theta')$. From induction on ϕ , b_{ϕ_2} is accepting. Moreover, b is also accepting because control locations of the form $[p', \phi]$ are accepting. If s is finite, it either ends with a *ret* statement, or an infinite loop inside a called procedure. If *ret* statement ended s at s_j , then there is an infinite branch b with j nodes of the form $(\langle [p', \phi], \omega' \rangle, \theta')$, the children of the last being nodes of the form $(\langle [p', \phi_2], \omega' \rangle, \theta')$ and $(\langle True, \omega' \rangle, \theta')$. If an unfinished *call* ended s at s_j , then there is an infinite branch b with j nodes of the form $(\langle [p', \phi], \omega' \rangle, \theta')$, the children of the last being nodes of the form $(\langle [p', \phi_2], \omega' \rangle, \theta')$ and $(\langle [p', \phi]^r, \omega' \rangle, \theta')$. Then, the branch starting from $(\langle [p', \phi]^r, \omega' \rangle, \theta')$ is accepting because control locations of the form $[p', \phi]^r$ are accepting and repeat infinitely many times (because the call at s_j never returns).

- ($\Leftarrow$) The accepting run can only be constructed using rules from item 21 and 8. Let $s = s_0 s_1 \dots$ be the sequence such that $s_0 = i$ and for each $j \geq 0$, $s_{j+1} = next_{\pi'}^a(s_j)$. There are three cases:
 - * there is a branch b with infinitely many nodes of the form $(\langle [p', \phi], \omega' \rangle, \theta')$. Let b_j be the j -th node of such form. First, we show that $b_j = Assoc(\phi, \pi', s_j)$. By definition, $b_0 = Assoc(\phi, \phi', s_0)$. Then, for each $j \geq 0$, if b_{j+1} has the form $(\langle [p', \phi], \omega' \rangle, \theta')$, then there is $p\gamma \xrightarrow{(\rho, \rho')}_{int} p'w \in \tau$ and thus, $s_{j+1} = s_j + 1$. If the child of b_j has the form $(\langle [p', \phi]^r, \omega' \rangle, \theta')$, then there is $p\gamma \xrightarrow{(\rho, \rho')}_{call} p'w \in \tau$. Similar to the case 5, we get that b reaches b_{j+1} iff $b_j = Assoc(\phi, \pi', s_{j+1})$.
 - * There is the last b_j , such that the children of b_j are of the forms $(\langle [p', \phi_2], \omega' \rangle, \theta')$ and $(\langle True, \omega' \rangle, \theta')$. Then, the rule applied at s_j must be *ret*, and therefore, s also ends at s_j .
 - * There is the last b_j , such that the children of b_j are of the forms $(\langle [p', \phi_2], \omega' \rangle, \theta')$ and $(\langle [p', \phi]^r, \omega' \rangle, \theta')$. Then, the rule applied at s_j must be *call*. Since there is no more nodes of the form $(\langle [p', \phi], \omega' \rangle, \theta')$, then the corresponding return was not met, and therefore, s also ends at s_j .

Each node b_j has another child of the form $(\langle [p', \phi_2], \omega' \rangle, \theta')$. Thus, from the induction on ϕ , we get that for each s_j , $(\pi', s_j) \models \phi_2$.

14. $\phi = A[\phi_1 R^a \phi_2]$. The proof similar to case 13.

15. $\phi = EX^c \phi'$. We consider two cases:

- ($next_{\pi}^c(i)$ is defined) Suppose $next_{\pi}^c(i) = k$, $\pi(i) = (\langle p_i, w\omega \rangle, \theta_i)$, and $\pi(k) = (\langle p_k, \gamma_k \omega \rangle, \theta_k)$. Let the call at the k -th step would be of the form $p_k \gamma_k \xrightarrow{\sigma}_{call} p_{k+1} \gamma' \gamma''$. Hence, if $Assoc(\phi', \pi, k) = (\langle [p_k, \phi'] \gamma_k \omega' \rangle, \theta_k)$, then $Assoc(\phi, \pi, i)$ is of the form $(\langle [p_i, \phi], w' \omega' \rangle, \theta_i)$, s.t. $w = u \gamma''$ and $w' = \gamma_i u [\gamma'', p_k, \gamma_k]^{ \theta_k }$. Item 23 adds a rule of the form $[p_i, \phi] \gamma_i \xrightarrow{[\top]} \{[p^c, \phi] \varepsilon\}$ and item 26 adds rules of the form $[p^c, \phi] \gamma^j \xrightarrow{[\top]} \{[p^c, \phi] \varepsilon\}$ for $\gamma^j \in u$. Only these rules are allowed to be applied at each control location. Hence, $(\langle [p_i, \phi], w' \omega' \rangle, \theta_i) \Rightarrow^* \{(\langle [p^c, \phi], [\gamma'', p_k, \gamma_k]^{ \theta_k } \omega' \rangle, \theta_i)\}$. Then, only the rule from *restore* can be applied and it allows $(\langle [p^c, \phi], [\gamma'', p_k, \gamma_k]^{ \theta_k } \omega' \rangle, \theta_i) \Rightarrow \{(\langle [p_k, \phi'], \gamma_k \omega' \rangle, \theta_k)\}$.

By induction on ϕ , $(\pi, k) \models \phi'$ iff there is an accepting run from $(\langle [p_k, \phi'], \gamma_k \omega' \rangle, \theta_k) = Assoc(\phi', \pi, k)$ and hence, $(\pi, i) \models \phi$ iff there is an accepting run from $(\langle [p_i, \phi], w' \omega' \rangle, \theta_i) = Assoc(\phi, \pi, i)$.

- ($next_{\pi}^c(i)$ is undefined) Suppose $next_{\pi}^c(i) = \perp$. $Assoc(\phi, \pi, i)$ is of the form $(\langle [p_i, \phi], \gamma_i w \# \rangle, \theta_i)$, where $w \in \Gamma^*$. Item 23 adds a rule of the form $[p_i, \phi] \gamma_i \xrightarrow{[\top]} \{[p^c, \phi] \varepsilon\}$ and item 26 adds rules of the form $[p^c, \phi] \gamma^j \xrightarrow{[\top]} \{[p^c, \phi] \varepsilon\}$ for $\gamma^j \in w$. Only these rules can be applied at each control location. Thus, $(\langle [p_i, \phi], \gamma_i w \# \rangle, \theta_i) \Rightarrow^* \{(\langle [p^c, \phi], \# \rangle, \theta_i)\}$. Only rule of the form $[p^c, \phi] \# \xrightarrow{[\top]} \{[p^c, \phi] \# \}$ can

be applied, and since $[p^c, \phi] \notin F$, there is no accepting runs from $Assoc(\phi, \pi, i)$.

16. $\phi = E[\phi_1 U^c \phi_2]$. We prove in both directions:

- ($\implies$) Let $(\pi, i) \models \phi$, i.e. let $s = s_0 s_1 \dots s_k$ be the sequence, s.t. $s_0 = i$ and for $j \geq 0$, $s_{j+1} = next_\pi^c(s_j)$, and $(\pi, s_k) \models \phi_2$ and for $0 \leq j < k$, $(\pi, s_j) \models \phi_1$. By induction on ϕ , there are accepting runs from $Assoc(\phi_2, \pi, s_k)$ and each $Assoc(\phi_1, \pi, s_j)$. We proof by induction on k :
 - * **Base** $k = 0$, then $(\pi, i) \models \phi_2$, and then, there is an accepting run from $Assoc(\phi_2, \pi, i) = Assoc(\phi_2, \pi, s_k)$. The rule added by item 12 creates allows the accepting run starting from $Assoc(\phi, \pi, i) \implies \{Assoc(\phi_2, \pi, i)\}$.
 - * **Step** $k > 0$. Then, $(\pi, i) \models \phi_1$ and $(\pi, next_\pi^c(i)) \models \phi$. By induction on ϕ , there is an accepting from $Assoc(\phi_1, \pi, i)$, and by induction on k , there is an accepting run from $Assoc(\phi, \pi, next_\pi^c(i))$. Let $\pi(i) = (\langle p_i, \gamma_i u \gamma'' \omega \rangle, \theta_i)$, s.t. γ'' was pushed to the stack at the call locations using a rule of the form $p_{s_1} \gamma_{s_1} \xrightarrow{\sigma} p' \gamma' \gamma'' \in \tau$. Thus, $Assoc(\phi, \pi, i)$ is of the form $(\langle [p_i, \phi] \gamma_i u [\gamma'', p_{s_1}, \gamma_{s_1}] \omega' \rangle^{\theta_{s_1}}, \theta_i)$, where θ_{s_1} is the phase at the call location. Rule added by item 24 creates transition $Assoc(\phi, \pi, i) \implies \{Assoc(\phi_1, \pi, i), (\langle [p^c, \phi] u [\gamma'', p_{s_1}, \gamma_{s_1}] \omega' \rangle, \theta_i)\}$. Then, rules added by item 26 allow $(\langle [p^c, \phi] u [\gamma'', p_{s_1}, \gamma_{s_1}] \omega' \rangle, \theta_i) \implies^* \{(\langle [p^c, \phi] [\gamma'', p_{s_1}, \gamma_{s_1}] \omega' \rangle, \theta_i)\}$. Then using the *restore* rules: $(\langle [p^c, \phi] [\gamma'', p_{s_1}, \gamma_{s_1}] \omega' \rangle, \theta_i) \implies \{(\langle [p_{s_1}, \phi] \gamma_{s_1} \omega' \rangle, \theta_{s_1})\}$. And thus, $Assoc(\phi, \pi, i) \implies \{Assoc(\phi_1, \pi, i), Assoc(\phi, \pi, next_\pi^c(i))\}$.

And thus, there is an accepting run from $Assoc(\phi, \pi, i)$.

- ($\impliedby$) Let $Assoc(\phi, \pi, i) = (\langle [p_i, \phi] \omega \rangle, \theta_i)$. Let k be the number of checkpoints $\chi^\theta \in G$ in ω . We prove by induction on k .
 - * **Base** $k = 0$. Then, only transition $Assoc(\phi, \pi, i) \implies \{Assoc(\phi_2, \pi, i)\}$ is possible. By induction on ϕ , $(\pi, i) \models \phi_2$ and thus, $(\pi, i) \models \phi$.
 - * **Step** $k > 0$. Then, either base case applies, or the accepting run starts with $Assoc(\phi, \pi, i) \implies \{Assoc(\phi_1, \pi, i), (\langle [p^c, \phi] u [\gamma'', p_{s_1}, \gamma_{s_1}]^{\theta_{s_1}} \omega' \rangle, \theta_i)\} \implies^* \{Assoc(\phi_1, \pi, i), (\langle [p_{s_1}, \phi] \gamma_{s_1} \omega' \rangle, \theta_{s_1})\}$, where $\omega = \gamma u [\gamma'', p_{s_1}, \gamma_{s_1}]^{\theta_{s_1}} \omega'$. Since $(\langle [p_{s_1}, \phi] \gamma_{s_1} \omega' \rangle, \theta_{s_1}) = Assoc(\phi, \pi, next_\pi^c(i))$ uses less checkpoints, by induction on k , $(\pi, next_\pi^c(i)) \models \phi$. Also, by induction on ϕ , $(\pi, i) \models \phi_1$. Thus, $(\pi, i) \models \phi$.

17. $\phi = E[\phi_1 R^c \phi_2]$. Let $s = s_0 s_1 \dots s_k$, s.t. $s_0 = i$, for $j \geq 0$, $s_{j+1} = next_\pi^c(s_j)$. We prove in both directions:

- ($\implies$) Let $(\pi, i) \models \phi$. There are two cases: either there is j , s.t. $(\pi, s_j) \models \phi_1 \wedge \phi_2$, and for each $0 \leq l < j$, $(\pi, s_l) \models \phi_1$. In this case, the proof is similar to case 16. Or, $\forall j, 0 \leq j \leq k$, $(\pi, s_j) \models \phi_2$. In this case, we prove by induction on k :
 - * **Base** $k = 0$. Then, $next_\pi^c(i) = \perp$ and if $\pi(i) = (\langle p, \gamma w \rangle, \theta)$, then w does not contain any return locations pushed by *call* rules. Hence, $Assoc(\phi, \pi, i) = (\langle [p, \phi], \gamma w \# \rangle, \theta)$. There is a rule created by item 25, which allows $Assoc(\phi, \pi, i) \implies \{Assoc(\phi_2, \pi, i), (\langle [p^c, \phi] w \# \rangle, \theta)\}$.

And rules added by item 26 allow $(\langle [p^c, \phi]w\# \rangle, \theta) \Rightarrow^* \{(\langle [p^c, \phi]\# \rangle, \theta)\}$.

By induction on ϕ , there is an accepting run from $Assoc(\phi_2, \pi, i)$.

And there is an accepting run from $(\langle [p^c, \phi]\# \rangle, \theta)$ created by the rule added by item 26 because $[p^c, \phi] \in F^c \subseteq F$.

- * **Step** $k > 0$. By induction on ϕ , there is an accepting run from $Assoc(\phi_2, \pi, i)$. Let $\pi(i) = (\langle p_i, \gamma_i u \gamma'' \omega \rangle, \theta_i)$, s.t. γ'' was pushed to the stack at the call locations using a rule of the form $p_{s_1} \gamma_{s_1} \xrightarrow{\sigma} p' \gamma' \gamma'' \in \tau$. Thus, $Assoc(\phi, \pi, i)$ is of the form $(\langle [p_i, \phi] \gamma_i u \lfloor \gamma'' \rfloor_{p_{s_1}, \gamma_{s_1}} \omega' \rangle^{\theta_{s_1}}, \theta_i)$, where θ_{s_1} is the phase at the call location. Rule added by item 25 creates transition $Assoc(\phi, \pi, i) \Rightarrow \{Assoc(\phi_2, \pi, i), (\langle [p^c, \phi] u \lfloor \gamma'' \rfloor_{p_{s_1}, \gamma_{s_1}} \omega'), \theta_i)\}$.

Then, rules added by item 26 allow $(\langle [p^c, \phi] u \lfloor \gamma'' \rfloor_{p_{s_1}, \gamma_{s_1}} \omega' \rangle, \theta_i) \Rightarrow^*$

$\{(\langle [p^c, \phi] \lfloor \gamma'' \rfloor_{p_{s_1}, \gamma_{s_1}} \omega' \rangle, \theta_i)\}$. Then using the *restore* rules: $(\langle [p^c, \phi] \lfloor \gamma'' \rfloor_{p_{s_1}, \gamma_{s_1}} \omega' \rangle, \theta_i) \Rightarrow$

$\{(\langle [p_{s_1}, \phi] \gamma_{s_1} \omega' \rangle, \theta_{s_1})\}$. By induction on k , there is an accepting

run from $Assoc(\phi, \pi, next_\pi^c(i)) = (\langle [p_{s_1}, \phi] \gamma_{s_1} \omega' \rangle, \theta_{s_1})$, and since $Assoc(\phi, \pi, i) \Rightarrow \{Assoc(\phi_2, \pi, i), Assoc(\phi, \pi, next_\pi^c(i))\}$, then

there is an accepting run from $Assoc(\phi, \pi, i)$.

- ($\Leftarrow$) There are two cases: either $Assoc(\phi, \pi, i)$ reaches some configuration $\{Assoc(\phi_1, \pi, s_j), Assoc(\phi_2, \pi, s_j)\}$. In this case, the proof is similar to case 16. Otherwise, the run reaches configurations with nodes $\{Assoc(\phi_1, \pi, s_k), (\langle [p_{s_k}, \phi]\# \rangle, \theta_{s_k})\}$. In this case, the accepting run has the form of a tree with one branch containing k nodes of the form $(\langle [p_{s_j}, \phi] \omega' \rangle, \theta_{s_j})$, and the k -th such node has children $Assoc(\phi_2, \pi, s_k)$ and $(\langle [p^c, \phi] u \# \rangle, \theta_{s_k})$. Moreover, for each s_j , there is a child of the form $Assoc(\phi_2, \pi, s_j)$ (created by the rule added by item 25). By induction on ϕ , $(\pi, s_j) \models \phi_2$ for each $0 \leq j \leq k$, and therefore, $(\pi, i) \models \phi$.

B Proof of Theorem 2

Proof. First, we prove the following lemma:

Lemma 1. $\mathcal{BP}$ has a run η starting from c , such that each path visits accepting control locations at least k times iff $c \in Y_k$.

Proof. We prove in both directions:

- ($\Rightarrow$) We prove by induction on k . Base case is trivial since $Y_0 = Conf_{\mathcal{BP}}$. Let $c \Rightarrow^* \{(\langle p_1, w_1 \rangle, \theta_1), \dots, (\langle p_n, w_n \rangle, \theta_n)\}$, s.t. for $1 \leq i \leq n$, $(\langle p_i, w_i \rangle, \theta_i)$ is the first visited accepting node ($p \in F$). Then, for $1 \leq i \leq n$, $\mathcal{BP}$ has a run η_i starting from $(\langle p_i, w_i \rangle, \theta_i)$ and visiting accepting control locations at least $k-1$ times. By induction hypothesis, $(\langle p_i, w_i \rangle, \theta_i) \in Y_{k-1}$. Since $Y_{k-1} \cap (F \times G^* \times 2^\delta) \supseteq \{(\langle p_1, w_1 \rangle, \theta_1), \dots, (\langle p_n, w_n \rangle, \theta_n)\}$, then $c \in pre^*(Y_{k-1} \cap (F \times G^* \times 2^\delta)) = Y_k$.

- ($\Leftarrow$) We prove by induction on k . Base case is trivial since $Y_0 = Conf_{\mathcal{BP}}$. Since $Y_k = pre^*(Y_{k-1} \cap (F \times G^* \times 2^\delta))$, then $c \Rightarrow^* \{(\langle p_1, w_1 \rangle, \theta_1), \dots, (\langle p_n, w_n \rangle, \theta_n)\}$, where $\{(\langle p_1, w_1 \rangle, \theta_1), \dots, (\langle p_n, w_n \rangle, \theta_n)\} \subseteq Y_{k-1} \cap (F \times G^* \times 2^\delta)$, i.e. for each $1 \leq i \leq n$, $p_i \in F$ and $(\langle p_i, w_i \rangle, \theta_i) \in Y_{k-1}$. By induction hypothesis, there is a run from $(\langle p_i, w_i \rangle, \theta_i)$ visiting accepting states at least $k-1$ times. And since $p_i \in F$, then there is a run η from c that visits accepting states at least k times.

Now we proceed with the proof of Theorem 2. We prove in both directions:

- ($\Rightarrow$) We show by proving that if the configuration c is not in $Y_{\mathcal{BP}}$, then there is no accepting runs from c in $\mathcal{BP}$. Suppose $c \notin Y_{\mathcal{BP}}$. Thus, there exists Y_k , s.t. $c \notin Y_k$. From Lemma 1, there is no run starting from c and visiting accepting control locations at least k times. Thus, there is no accepting runs from c .
- ($\Leftarrow$) Suppose $c \in Y_{\mathcal{BP}}$. It is sufficient to construct an accepting run starting from c . Since the sequence $Y_0 Y_1 \dots$ is monotone and $Y_{\mathcal{BP}} = \bigcap_{j \geq 0} Y_j$, we obtain that $Y_{\mathcal{BP}} = pre^+(Y_{\mathcal{BP}} \cap F \times G^* \times 2^\delta)$. Since $c \in pre^+(Y_{\mathcal{BP}} \cap F \times G^* \times 2^\delta)$, then there is a set of configurations $\{c_1, \dots, c_n\} \subseteq Y_{\mathcal{BP}} \cap F \times G^* \times 2^\delta$, s.t. $c \Rightarrow^* \{c_1, \dots, c_n\}$. Therefore, for each $1 \leq i \leq n$, $c_i \in Y_{\mathcal{BP}}$.

In other words, for $c \in Y_{\mathcal{BP}}$, we can construct a finite subrun η in $\mathcal{BP}$ with root c and leaves $\{c_1, \dots, c_n\}$, s.t. for each $1 \leq i \leq n$, $c_i \in Y_{\mathcal{BP}}$. Then, we can extend each c_i in η with a new subrun η_i that starts at c_i and each leaf of η_i is also in $Y_{\mathcal{BP}}$. We can repeat this process infinitely many times, thus obtaining the accepting run η .

C Proof of Theorem 3

Proof. The proof is mostly identical to the proof of Theorem 2 in [23]. However, we use a different ABPDS model and hence, we need to prove their Lemma 3 and Lemma 4 for SM-ABPDS.

Lemma 2 corresponds to Lemma 3 in [23]:

Lemma 2. *In Algorithm 1, for every $i \geq 1$, after line 18, we have that $\mathcal{L}(A_i) = pre^+(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$.*

Proof. We prove in both directions.

$(L(A_i) \subseteq pre^+(L(A_{i-1}) \cap (F \times G^* \times 2^\delta)))$: Suppose $(\langle p, w \rangle, \theta) \in A_i$. There are no transitions of the form $(p, \theta) \xrightarrow{\varepsilon} \{q_f\}$ after line 18 therefore, $|w| \geq 1$.

We can show that during *loop*₂, if $(\langle p, w \rangle, \theta) \in A_i$, then $(\langle p, w \rangle, \theta) \in pre^+(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$. We prove by induction on w .

Base $w = \varepsilon$. Then, we have a transition added at line 4, which means $p \in F$ and $(p, \theta)^i \xrightarrow{\varepsilon} \{(p, \theta)^{i-1}\}$. Therefore, respectively, $(\langle p, w \rangle, \theta) \in F \times G^* \times 2^\delta$ and $(\langle p, w \rangle, \theta) \in L(A_{i-1})$. And therefore, the base case holds.

Step There are two cases:

- $w = \gamma u$ for $\gamma \in \Gamma, u \in G^*$. Thus, there exist $S \subseteq Q$, s.t. $t = (p, \theta)^i \xrightarrow{\gamma} S \in T$, $S \xrightarrow{u}^* \{q_f\}$. t could be added to T at the only one place on line 9, which means $\exists p\gamma \xrightarrow{[\sigma_1, \dots, \sigma_m]} \{p_1 w_1, \dots, p_m w_m\}$, $\sigma_j = (\rho_j, \rho'_j)$, $S = \{q \in S_j \mid \rho_j \subseteq \theta \text{ and } (p_j, \theta_j)^l \xrightarrow{w_j}^* S_j\}$, where $\theta_j = (\theta \setminus \rho_j) \cup \rho'_j$. By applying the induction hypothesis, for every j , $1 \leq j \leq m$, if $\rho_j \subseteq \theta$, then $(\langle p_j, w_j u \rangle, \theta_j) \in pre^*(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$. Thus, we get that $(\langle p, w \rangle, \theta) \in pre^*(L(A_{i-1}) \cap (F \times G_i^* \times 2^\delta))$.
- $w = \chi^{\theta'} u$ for $\chi \in X, u \in G^*, \theta' \subseteq \delta$. Thus, there exist $S \subseteq Q$, s.t. $t = (p, \theta)^i \xrightarrow{\chi^{\theta'}} S \in T$, $S \xrightarrow{u}^* \{q_f\}$. t could be added to T at two places:
 - On line 12, which means $\exists p\chi \hookrightarrow p'\gamma \in restore$, $(p', \theta')^l \xrightarrow{\gamma}^* S$. By applying the previous item, we have that $(\langle p', \gamma u \rangle, \theta') \in pre^*(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$. Thus, we get that $(\langle p, w \rangle, \theta) \in pre^*(L(A_{i-1}) \cap (F \times G_i^* \times 2^\delta))$.
 - On line 15, which means $\exists p\chi \hookrightarrow p'\gamma \in discard$, $(p', \theta')^l \xrightarrow{\gamma}^* S$. By applying the previous item, we have that $(\langle p', \gamma u \rangle, \theta) \in pre^*(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$. Thus, we get that $(\langle p, w \rangle, \theta) \in pre^*(L(A_{i-1}) \cap (F \times G_i^* \times 2^\delta))$.

Since the algorithm removes ε transitions on line 18, we know that $|\omega| \geq 1$. We can apply the same induction but with at least one step to obtain $(\langle p, \omega \rangle, \theta) \in pre^+(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$.

$(pre^+(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))) \subseteq L(A_i)$:

Since $pre^+(W) = pre^*(pre(W))$ for any $W \subseteq Conf$, we obtain that $pre^+(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$ is the limit of the infinite sequence $\{C_j\}_{j \geq 0}$, such that $C_0 = pre(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$ and $C_{j+1} = C_j \cup pre(C_j)$ for every j , $j \geq 0$. Because $C_j \subseteq C_{j+1}$ for every $j \geq 0$, it is sufficient to show that for every tuple $(\langle p, w \rangle, \theta) \in C_j$, we have a path $(p, \theta) \xrightarrow{w}^* \{q_f\}$ in A_i that does not use transition rules of the form $q^i \xrightarrow{\varepsilon} \{q^{i-1}\}$ for any $q^i, q^{i-1} \in Q$. We prove by induction.

Base $j = 0$. By applying the saturation procedure at $loop_2$, A_i accepts C_0 if only the states of the form $(p, \theta)^i$ are treated as initial states, which are the outgoing states of the added transitions during $loop_2$. Since these transitions are of the form $(p, \theta)^i \xrightarrow{\gamma} S$ for $\gamma \in G$, $Q \subseteq P \times 2^\delta \times \{i-1\} \cup \{q_f\}$, we have that for every $(\langle p, w \rangle, \theta) \in C_0$, $(\langle p, w \rangle, \theta) \in L(A_i)$.

Step $j \geq 1$. Since $C_j = C_{j-1} \cup pre(C_{j-1})$, we get that either $(\langle p, w \rangle, \theta) \in C_{j-1}$ or $(\langle p, w \rangle, \theta) \in pre(C_{j-1})$. In the first case, the step holds from the induction hypothesis. Otherwise, if $(\langle p, w \rangle, \theta) \in pre(C_{j-1})$, there are two cases depending on what rule was applied:

- $\exists p\gamma \xrightarrow{[\sigma_1, \dots, \sigma_m]} \{p_1 w_m, \dots, p_m w_m\} \in \Delta'$, such that $w = \gamma u$ and for every j , $1 \leq k \leq m$, let $\sigma_k = (\rho_k, \rho'_k)$, if $\rho_k \subseteq \theta$, then $(\langle p_k, em(\theta, w_k)u \rangle, \theta_k) \in C_{j-1}$, where $\theta_k = (\theta \setminus \rho_k) \cup \rho'_k$. We apply the induction hypothesis to get $(p_k, \theta_k)^i \xrightarrow{em(\theta, w_k)u}^* \{q_f\}$. Let us deconstruct the path as follows: $(p_k, \theta_k)^i \xrightarrow{em(\theta, w_k)}^* Q_j \xrightarrow{u}^* \{q_f\}$. Therefore, the saturation procedure will add the transition

- $(p, \theta) \xrightarrow{\gamma} S$, where $S = \{q \in Q_k \mid \rho_k \subseteq \theta\}$. Thus, we get that $(\langle p, \rangle, \theta) \in L(A_i)$.
- $\exists p\chi \hookrightarrow p'\gamma \in \text{restore}$, $w = \chi^{\theta'}u$, $(\langle p', \gamma u \rangle, \theta') \in C_{j-1}$ for some $\theta' \subseteq \delta$. We apply the induction hypothesis to get $(p', \theta')^i \xrightarrow{\gamma^u}^* \{q_f\}$. Let us deconstruct the path as follows: $(p', \theta')^i \xrightarrow{\gamma}^* S \xrightarrow{u}^* \{q_f\}$. Therefore, the saturation procedure will add the transition $(p, \theta) \xrightarrow{\chi^{\theta'}} S$. Thus, we get that $(\langle p, w \rangle, \theta) \in L(A_i)$.
 - $\exists p\chi \hookrightarrow p'\gamma \in \text{discard}$, $w = \chi^{\theta'}u$, for some $\theta' \subseteq \delta$, and $(\langle p', \gamma u \rangle, \theta) \in C_{j-1}$. We apply the induction hypothesis to get $(p', \theta')^i \xrightarrow{\gamma^u}^* \{q_f\}$. Let us deconstruct the path as follows: $(p', \theta')^i \xrightarrow{\gamma}^* S \xrightarrow{u}^* \{q_f\}$. Therefore, the saturation procedure will add the transition $(p, \theta) \xrightarrow{\chi^{\theta'}} S$. Thus, we get that $(\langle p, w \rangle, \theta) \in L(A_i)$.

Therefore, $\text{pre}^+(L(A_{i-1}) \cap (F \times G^* \times 2^\delta)) \subseteq L(A_i)$.

Lemma 3 corresponds to Lemma 4 in [23]:

Lemma 3. *Consider an **Algorithm C**, which is Algorithm 1 without line 20 (without termination condition of loop₁). In **Algorithm C**, for every $i \geq 0$:*

- (a) *for every accepting run η in $\mathcal{BP}$ starting from $(\langle p, w \rangle, \theta)$, there is a path $(p, \theta)^i \xrightarrow{w}^* \{q_f\}$ in A_i and for every decomposition $(p, \theta)^i \xrightarrow{u} Q \xrightarrow{v} \{q_f\}$ of the path, if $Q \neq \{q_f\}$, then for all $(p', \theta')^i \in Q \setminus \{q_f\}$, η visits $(\langle p', v \rangle, \theta')$;*
- (b) *After substitution at line 19, $Y_{\mathcal{BP}} \subseteq L(A_i)$.*

Proof. We proceed by induction on i .

Base $i = 0$. The statement (a) holds from the fact that for any $p \in P'$, $\theta \subseteq \delta$, $w \in G^*$, $(p, \theta)^0 \xrightarrow{w}^* \{q_f\}$ during $i = 0$ and for any decomposition $(p, \theta)^i \xrightarrow{u}^* Q \xrightarrow{v}^* \{q_f\}$, $Q = \{q_f\}$. The statement (b) holds from the fact that $\mathcal{L}(A_0) = P' \times G^* \times 2^\delta$.

Step $i \geq 1$. For the statement (a). Let $H(\eta)$ be the maximum number of steps over branches of η to reach some accepting control location. We proceed by induction on $H(\eta)$.

- **Base** $H(\eta) = 0$. Since the root of η is $(\langle p, w \rangle, \theta)$, then we have that $(\langle p, w \rangle, \theta) \in F \times G^* \times 2^\delta$. The line 1 adds transition $(p, \theta)^i \xrightarrow{\varepsilon} \{(p, \theta)^{i-1}\}$ and the result follows from the induction hypothesis on i .
- **Step** $H(\eta) \geq 1$. Let the root node of η , $(\langle p, w \rangle, \theta)$, have m child runs $\eta_1, \dots, \eta_m$, s.t. for every j , $1 \leq j \leq m$, η_j starts at $(\langle p_j, w_j \rangle, \theta_j)$. There are three cases depending on which rule was applied:
 - $\exists p\gamma \xrightarrow{[\sigma_{j_1}, \dots, \sigma_{j_h}]} \{p_{j_1}\omega_{j_1}, \dots, p_{j_h}\omega_{j_h}\} \triangleright \{D_{j_1}, \dots, D_{j_h}\}$, s.t. $\{1, \dots, m\} \subseteq \{j_1, \dots, j_h\}$, $w = \gamma w'$, and for every j , $1 \leq j \leq m$, $\theta_j = (\theta \setminus \rho_j) \cup \rho'_j$, $\sigma_j = (\rho_j, \rho'_j)$, and $\rho_j \subseteq \theta$. We apply the induction on $H(\eta_j)$ for every j , $1 \leq j \leq m$, to obtain that there is a path $(p_j, \theta_j)^i \xrightarrow{em(\theta, w_j)w'}^* \{q_f\}$ in

A_i and for every decomposition $(p_j, \theta_j)^i \xrightarrow{u} Q_j \xrightarrow{v} \{q_f\}$, if $Q_j \neq \{q_f\}$, then for every $(p', \theta')^i$ or $(p', \theta')^{i-1} \in Q_j \setminus \{q_f\}$, η_j reaches a node labelled $(\langle p', v \rangle, \theta')$.

Let $(p_j, \theta_j)^i \xrightarrow{w_j} Q'_j \xrightarrow{w'} \{q_f\}$. After application of the saturation at line 9, we get:

$$(p, \theta)^i \xrightarrow{\gamma} \bigcup_{j=1}^m Q'_j \xrightarrow{w'} \{q_f\}$$

For every decomposition $(p, \theta)^i \xrightarrow{u} Q \xrightarrow{v} \{q_f\}$, $Q \subseteq \bigcup_{j=1}^m Q_j$, which means that for every state of the form $(p', \theta')^i$ or $(p', \theta')^{i-1} \in Q \setminus \{q_f\}$, η reaches a node labelled $(\langle p', v \rangle, \theta')$.

- $\exists p\chi \hookrightarrow p_1\gamma \in \text{restore}$, s.t. $m = 1$, $w = \chi^{\theta'}\omega$ for some θ' , and $(\langle p_1, \gamma\omega \rangle, \theta')$ is the only child node. We apply the induction hypothesis on $H(\eta_1)$ to obtain that there is a path $(p_1, \theta')^i \xrightarrow{\gamma\omega} \{q_f\}$ in A_i , and for every decomposition of the form $(p_1, \theta')^i \xrightarrow{u} Q \xrightarrow{v} \{q_f\}$, if $Q \neq \{q_f\}$, then for every $(p', \theta'')^i$ or $(p', \theta'')^{i-1} \in Q \setminus \{q_f\}$, η_1 reaches a node labelled $(\langle p', v \rangle, \theta'')$.

Let $(p_1, \theta')^i \xrightarrow{\gamma} Q' \xrightarrow{\omega} \{q_f\}$. After application of the saturation at line 9, we get:

$$(p, \theta)^i \xrightarrow{\chi^{\theta'}} Q' \xrightarrow{\omega} \{q_f\}$$

Therefore, since $(p, \theta)^i \xrightarrow{w} \{q_f\}$, we have that for every decomposition $(p, \theta)^i \xrightarrow{u} Q \xrightarrow{v} \{q_f\}$, $Q \subseteq \bigcup_{j=1}^m Q_j$, which means that for every state of the form $(p', \theta'')^i$ or $(p', \theta'')^{i-1} \in Q \setminus \{q_f\}$, η reaches a node labelled $(\langle p', v \rangle, \theta'')$.

- $\exists p\chi \hookrightarrow p_1\gamma \in \text{discard}$, s.t. $m = 1$, $w = \chi^{\theta'}\omega$ for some θ' , and $(\langle p_1, \gamma\omega \rangle, \theta)$ is the only child node. We apply the induction hypothesis on $H(\eta_1)$ to obtain that there is a path $(p_1, \theta)^i \xrightarrow{\gamma\omega} \{q_f\}$ in A_i , and for every decomposition of the form $(p_1, \theta)^i \xrightarrow{u} Q \xrightarrow{v} \{q_f\}$, if $Q \neq \{q_f\}$, then for every $(p', \theta'')^i$ or $(p', \theta'')^{i-1} \in Q \setminus \{q_f\}$, η_1 reaches a node labelled $(\langle p', v \rangle, \theta'')$.

Let $(p_1, \theta)^i \xrightarrow{\gamma} Q' \xrightarrow{\omega} \{q_f\}$. After application of the saturation at line 9, we get:

$$(p, \theta)^i \xrightarrow{\chi^{\theta'}} Q' \xrightarrow{\omega} \{q_f\}$$

Therefore, since $(p, \theta)^i \xrightarrow{w} \{q_f\}$, we have that for every decomposition $(p, \theta)^i \xrightarrow{u} Q \xrightarrow{v} \{q_f\}$, $Q \subseteq \bigcup_{j=1}^m Q_j$, which means that for every state of the form $(p', \theta'')^i$ or $(p', \theta'')^{i-1} \in Q \setminus \{q_f\}$, η reaches a node labelled $(\langle p', v \rangle, \theta'')$.

Thus, the statement (a) holds.

For the statement (b). First, we need to show that $Y_{\mathcal{BP}} \subseteq L(A_1)$. From Lemma 2, we have that $L(A_1) = \text{pre}^+(L(A_0) \cap F \times G^* \times 2^\delta)$, i.e. $L(A_1) = \text{pre}^+(F \times G^* \times 2^\delta)$ before line 19. Since $Y_1 = \text{pre}^+(F \times G^* \times 2^\delta)$, it is sufficient

to prove that for every $k \geq 1$, $(\langle p, w \rangle, \theta) \in Y_{\mathcal{BP}}$, there is a path $(p, \theta)^k \xrightarrow{w^*} \{q_f\}$ in A_k after line 19. This is proven in what follows.

Suppose $k \geq 2$. We apply the induction hypothesis on k to obtain that $Y_{\mathcal{BP}} \subseteq L(A_{k-1})$.

Since $Y_{\mathcal{BP}} = pre^+(Y_{\mathcal{BP}} \cap F \times G^* \times 2^\delta)$, we get that $Y_{\mathcal{BP}} \subseteq pre^+(L(A_{k-1}) \cap F \times G^* \times 2^\delta)$. From Lemma 2, $L(A_k) = pre^+(L(A_{k-1}) \cap F \times G^* \times 2^\delta)$ after line 18. Therefore, we prove that for every $k \geq 1$, if $(\langle p, w \rangle, \theta) \in Y_{\mathcal{BP}}$, then there is a path $(p, \theta)^k \xrightarrow{w^*} \{q_f\}$ in A_k after line 19. Let m be the number of transitions substituted at line 19. For l , $0 \leq l \leq m$, let A_k^l be the automaton obtained after substituting l transitions. We proceed by induction on l .

- **Base** $l = 0$. We directly get that $Y_{\mathcal{BP}} \subseteq A_k^0$.
- **Step** $l \geq 1$. We apply the induction hypothesis to obtain that $Y_{\mathcal{BP}} \subseteq L(A_k^{l-1})$. If $L(A_k^{l-1}) \subseteq L(A_k^l)$, the item holds from the fact that $Y_{\mathcal{BP}} \subseteq A_k^{l-1}$. Otherwise, suppose there is $(\langle p, \omega \rangle, \theta) \in L(A_k^{l-1}) \setminus L(A_k^l)$, such that $|\omega| = \min\{|\omega'| \mid (\langle p', \omega' \rangle, \theta') \in L(A_k^{l-1}) \setminus L(A_k^l)\}$. We prove by contradiction that $(\langle p, \omega \rangle, \theta) \notin Y_{\mathcal{BP}}$.
 Since $(p, \theta)^k \xrightarrow{w^*} \{q_f\}$ in A_k^{l-1} , let $u \in G^+$, $v \in G^*$, $p' \in P'$, $\theta' \subseteq \delta$, $Q' \subseteq Q$, such that $w = uv$, $(p, \theta) \xrightarrow{u^*} Q' \cup \{(p', \theta')^{k-1}\} \xrightarrow{v^*} \{q_f\}$ in A_k^{k-1} , and there is no such path $(p', \theta')^k \xrightarrow{v^*} \{q_f\}$ in A_k^l (otherwise, $(\langle p, \omega \rangle, \theta) \in L(A_k^l)$). By the statement (a), the accepting run must reach the local configuration $(\langle p', v \rangle, \theta')$ during the accepting run. It is sufficient to show that $(\langle p', v \rangle, \theta') \notin Y_{\mathcal{BP}}$. Since $(\langle p', v \rangle, \theta') \in L(A_k^{m-1})$, then we have that $(\langle p', v \rangle, \theta') \in L(A_k^{m-1}) \setminus L(A_k^m)$. Since $loop_2$ can only add new transitions, we get that $|v| > |w|$, which contradicts the definition of $|w|$.

Thus, the statement (b) holds.